"Analysis, Detection and Mitigation of HTTP Request Smuggling Vulnerability"

A Report submitted

in partial fulfilment for the Degree of

MASTER OF TECHNOLOGY

IN

Cyber Security

Submitted By

VIPIN BIHARI

(01230030000600 2058)

Under the Supervision of

MR. PRAKASH KHASOR

(ASSISTANT PROFESSOR)

Submitted to



SCHOOL OF CYBER SECURITY & DIGITAL FORENSICS,

NATIONAL FORENSIC SCIENCES UNIVERSITY

GANDHINAGAR – 382007, GUJARAT, INDIA.

MAY, 2025

DECLARATION

I "**VIPIN BIHARI"** having Enrolment Number **"012300300006002058"** hereby declare that

a. The work contained in the dissertation report entitled **"Analysis, Detection and Mitigation of HTTP Request Smuggling Vulnerability"** is being submitted in partial fulfilment for the award of the degree of **"M. Tech. Cyber Security"** to **School of Cyber Security & Digital Forensics** is an authentic record of my own work done under the supervision of **"MR. PRAKASH KHASOR"**.

b. The work has not been submitted to any other Institute/ School / University for any degree or diploma.

c. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the School.

d. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the dissertation and giving their details in the references.

e. Whenever I have quoted written materials from other sources and due credit is given to the sources by citing them.

f. From the plagiarism test, it is found that the similarity index of whole dissertation is less than 10 % as per the university guidelines.

Date:

Place:

Signature of Student

Signature & Date
Supervisor

CERTIFICATE

This is to certify that the work contained in the dissertation entitled **"Analysis, Detection and Mitigation of HTTP Request Smuggling Vulnerability"**, submitted by **Vipin Bihari (Enroll. No.: 012300300006002058)** in partial fulfilment of the requirement for the award of the degree of **M. Tech. Cyber Security** to the **National Forensic Sciences University**, **Gandhinagar, Gujarat** is a record of bonafide work carried out by him under the direct my direct supervision and guidance.

Date:
Place:

Supervised By:

_____

(Mr. Prakash Khasor)
Assistant Professor,
School of Cyber Security and Digital Forensics,
National Forensic Sciences University,
Gandhinagar, India, 382007

_____

Dean, School of Cyber Security and Digital Forensics,
National Forensic Sciences University,
Gandhinagar, India, 382007

ACKNOWLEDGEMENTS

I would like to extend my sincere gratitude to **Mr. Prakash Khasor**, my guide at the *School of Cyber Security & Digital Forensics, National Forensic Sciences University*, for his unwavering support, insightful guidance, and expert feedback throughout the course of this project. His dedication and thoughtful suggestions were invaluable in shaping the direction and depth of this research.

I am also deeply appreciative of the faculty members at the *School of Cyber Security & Digital Forensics*, whose discussions, lectures, and encouragement helped refine my understanding and approach. Their collective expertise created an enriching academic environment that fostered curiosity and critical thinking.

Lastly, I offer my heartfelt thanks to my family and friends for their unwavering encouragement and understanding. Their belief in my work and constant moral support kept me motivated, especially during challenging phases of this project.

The successful completion of this research on the "**Analysis, Detection, and Mitigation of HTTP Request Smuggling Vulnerability**" is the result of collective efforts and guidance from all these individuals. I am deeply grateful for the role each one has played in making this journey fruitful and rewarding.

With Sincere Regards,

Vipin Bihari

M. Tech. Cyber Security

ABSTRACT

This report provides a comprehensive examination of the analysis, detection, and mitigation of HTTP Request Smuggling (HRS) vulnerabilities, which represent a significant security risk in web applications. These vulnerabilities arise due to discrepancies in the interpretation of HTTP requests by various intermediary systems.

The research delves into the technical complexities associated with HRS, beginning with a thorough overview of the HTTP protocol's evolution, contemporary web architectures, and the intricacies involved in parsing HTTP requests. It systematically analyzes multiple HRS attack vectors, specifically CL.TE, TE.CL, CL.CL, TE.TE, CL.0, H2.0, H2.CL, and H2.TE, detailing each vector's unique exploitation techniques.

A key contribution of this project is the development of an automated detection tool named "HRS Detector." This tool is specifically engineered to simplify and expedite the detection and validation of HRS vulnerabilities. The report thoroughly describes the functionality and workflow of the HRS Detector, highlighting its configurable modes, intuitive graphical user interface (GUI), comprehensive logging capabilities, and an integrated repeater module for in-depth manual analysis.

Empirical results obtained through testing the HRS Detector on multiple sample applications demonstrate the practicality and effectiveness of the proposed detection strategy. The findings indicate that the HRS Detector, by integrating automated vulnerability scanning with robust manual verification options, significantly enhances the efficiency and accuracy of identifying and confirming HTTP Request Smuggling vulnerabilities.

.

The tool "HRS Detector" that is developed to support this research is available at GitHub
https://github.com/vipinbihari/hrs-detector

Keywords: *HTTP Request Smuggling, Web Security, Vulnerability Detection, Penetration Testing, HTTP Protocol, CL.TE, TE.CL, H2.TE, H2.CL, Automated Security Tools, HTTP Parsing, HRS Detector.*

LIST OF ABBREVIATIONS

Abbreviation	Description
CL	Content-Length
TE	Transfer-Encoding
HRS	HTTP Request Smuggling
HTTP	Hyper Text Transfer Protocol
H2	HTTP2
CDS	Client-Side Desync
H2c	HTTP2 Over Cleartext
TLS	Transport Layer Security

LIST OF TABLES

Table 1 CL.0 Detection Table 25
Table 2 Literature Review 30

LIST OF FIGURES

Figure 1 HTTP 1.0 6
Figure 2 HTTP 1.1 Keep-Alive 8
Figure 3 HTTP 1.1 Pipelining 9
Figure 4 Modern Frontend - Backend Architecture 13
Figure 5 HRS Illustration 17
Figure 6 HRS Raw Request 19
Figure 7 CL.TE Type HRS 20
Figure 8 TE.CL HRS 22
Figure 9 HRS Type CL.CL 23
Figure 10 HTTP2 to HTTP/1.1 Downgrade 25
Figure 11 H2.CL Type HRS 26
Figure 12 H2.TE Type HRS 27
Figure 13 High Level Tool Workflow 33
Figure 14 HRS Tool Flowchart 35
Figure 15 CL.TE Detection Raw Req 1 37
Figure 16 CL.TE Detection Raw Req 2 37
Figure 17 TE.CL Detection raw Req. 1 38
Figure 18 TE.CL Detection Raw Req. 2 39
Figure 19 HRS Detector Scan Configuration UI 48
Figure 20 HRS Detector Header Placements 49
Figure 21 HRS Detector Scan Results Console Area 50
Figure 22 HRS Detector Vulnerability Findings Table 51

TABLE OF CONTENTS

Table of Contents

DECLARATION ..... **II**

CERTIFICATE ..... **III**

ACKNOWLEDGEMENTS ..... **IV**

ABSTRACT ..... **V**

LIST OF ABBREVIATIONS ..... **VI**

LIST OF TABLES ..... **VII**

LIST OF FIGURES ..... **VIII**

TABLE OF CONTENTS ..... **IX**

1. Introduction **1**

1.1 Motivation 1

1.2 Scope of the Project 1

1.3 Organization of the Report 3

2. Theoretical Background & Literature Survey **4**

2.1 Introduction to HTTP Protocol 4

2.1.1 HTTP 1.X 5

HTTP 1.0 – The Early Web Era 5

HTTP 1.1 – A More Efficient and Flexible Web Protocol 6

HTTP Pipelining 8

2.1.2 HTTP 2 10

Understanding Pseudo-Headers in HTTP/2 11

Conceptual Example of an HTTP/2 Request 11

2.2 Modern Web Architecture 13

Frontend and Backend Servers 13

Key Concepts in a Layered Architecture 13

Example in Practice 14

2.3 Understanding HTTP Request Processing 15

The Structure of a Raw HTTP Request 15

Step-by-Step Parsing of a Raw HTTP Request 15

2.4 HTTP Request Smuggling Vulnerability 17

2.4.1 Definition and Concepts 17

2.4.2 Types of HTTP Request Smuggling Vulnerability....19
2.4.2.1 CL.TE....20
2.4.2.2 TE.CL....21
2.4.2.3 CL.CL....23
2.4.2.4 TE.TE....24
2.4.2.5 CL.0....24
2.4.2.6 H2.CL....25
2.4.2.7 H2.TE....27
2.4.3 Impact and Exploitation....28
2.4.4 Literature Review....30
3. The Proposed Model and Implementation Methodology....32
3.1 The Problem Statement....32
3.1.1 High Level workflow....33
3.1.2 HRS Detector workflow for CL.TE and TE.CL....35
Key Concepts in the Flow Chart....36
Step-by-Step Flow Explanation....36
3.1.3 HRS Detector workflow for H2.CL and H2.TE....40
H2.CL Detection....40
H2.TE Detection....41
4. Theoretical & Empirical Result Analysis....42
4.1 HRS Detector Overview....42
4.2 Tools and Technologies Used....44
4.3 Scanning Sample Applications....48
5. Conclusion....52
6. Future Scope....53
7. Bibliography....54
8. Plagiarism Report....55

1. Introduction

1.1 Motivation

The motivation behind the project titled "Analysis, Detection, and Mitigation of HTTP Request Smuggling Vulnerability" and the subsequent development of the HRS Detector tool stems from practical challenges frequently encountered by security professionals and developers. Today's web applications rely on a sophisticated interplay of services, including load balancers, proxies, and caching systems, to deliver enhanced performance and convenience. However, this intricate network introduces subtle vulnerabilities, creating opportunities for attackers to exploit weaknesses like HTTP Request Smuggling. Such attacks can bypass traditional security measures unnoticed, putting sensitive data, user confidence, and organizational reputations at significant risk.

For security teams tasked with safeguarding these systems, the complexity of modern web architectures often proves daunting. Detecting threats that are nuanced, difficult to identify, and continuously evolving adds considerable pressure. The responsibility placed on these individuals is substantial, as even a minor security oversight can result in severe consequences.

By addressing HTTP Request Smuggling specifically, and through the creation of the HRS Detector tool, this project seeks to support security professionals by providing a reliable and effective solution. The tool is designed to simplify the identification and remediation of vulnerabilities, thereby alleviating stress for security teams and offering organizations a more transparent and manageable approach to securing their web infrastructure. Ultimately, our objective is to foster a more predictable and secure online environment, transforming security operations from reactive guesswork into proactive management.

1.2 Scope of the Project

The project titled "Analysis, Detection, and Mitigation of HTTP Request Smuggling Vulnerabilities" specifically targets the identification and resolution of HTTP Request Smuggling (HRS) threats, a category of web vulnerabilities that arise due to discrepancies in how intermediary systems handle HTTP requests. The project's scope was thoughtfully designed to provide comprehensive coverage while remaining practically manageable. The primary objectives and boundaries defined within this project include:

1. **Theoretical Analysis:**
    - **Evolution of HTTP Protocol:** Examining the historical development of HTTP (versions 1.0, 1.1, and 2.0) to identify architectural and functional changes contributing to potential HRS vulnerabilities.

- **Modern Web Architectures:** Analyzing typical web application architectures, particularly the roles played by intermediaries like load balancers, reverse proxies, and content delivery networks (CDNs) in request handling and vulnerability exposure.
- **HTTP Request Parsing:** Conducting an in-depth review of HTTP request parsing mechanisms, emphasizing how headers (particularly Content-Length and Transfer-Encoding) and message bodies are interpreted and managed.
- **Attack Vector Analysis:** Identifying and examining major HRS attack vectors, including CL.TE, TE.CL, CL.CL, TE.TE, CL.0, H2.CL, and H2.TE, each assessed for their distinct exploit characteristics.

2. **Development of an Automated Detection Tool (HRS Detector):**
   - **Graphical User Interface (GUI):** Creating an intuitive GUI to facilitate ease of use, especially targeted towards penetration testers and security analysts.
   - **Scanning Functionalities:** Incorporating essential features such as handling inputs for single URLs or batches from files, configuration options for logging, choice of HTTP methods, and adjustable timeout settings.
   - **Detection Logic:** Implementing focused detection logic to assess server vulnerabilities by highlighting discrepancies in the interpretation of key HTTP headers (Content-Length and Transfer-Encoding) between frontend and backend systems.
   - **Integrated Repeater:** Providing an integrated manual request repeater to enable deeper analysis and validation through customizable resending of requests.
3. **Empirical Validation:**
   - **Sample Application Testing:** Deploying the HRS Detector on representative sample web applications to simulate realistic exploitation scenarios.
   - **Result Analysis:** Evaluating the effectiveness of detection capabilities through detailed analysis of the test results produced by the HRS Detector.

In conclusion, the project encompasses a detailed theoretical exploration of HRS vulnerabilities, the practical creation of an automated detection tool, and rigorous empirical validation. Although it does not exhaustively cover every aspect of HTTP Request Smuggling, it significantly contributes to the understanding and mitigation of this critical security concern, establishing a solid foundation for future research and advancements.

1.3 Organization of the Report

This report is structured to offer a logical and comprehensive examination of HTTP Request Smuggling (HRS) vulnerabilities, beginning with foundational concepts and progressing through the implementation and empirical validation of an automated detection tool. The organization of this report is as follows:

- **Chapter 1: Introduction**
  This chapter introduces the overall context of the report, detailing the background, motivation, and objectives of the project. It emphasizes the importance of addressing HRS vulnerabilities within modern web applications, outlines the project's scope, and explains the structure of the report.
- **Chapter 2: Theoretical Background & Literature Survey**
  This chapter provides the theoretical underpinnings necessary to understand HRS vulnerabilities. It covers the evolution and key features of HTTP protocols (HTTP 1.0, HTTP 1.1, and HTTP/2), explores modern web architectures and the role of intermediary components such as load balancers and reverse proxies, and delves deeply into the intricacies of HTTP request parsing. It also reviews existing literature and current methods for detecting HRS vulnerabilities, establishing a comprehensive background for the project.
- **Chapter 3: Proposed Model and Implementation Methodology**
  Here, the report outlines the proposed detection model and details the methodology for implementing the HRS Detector tool. This chapter clearly defines the identified issues, describes the approach taken to create the automated tool, and elaborates on the tool's key components, detection algorithm logic, and user workflows.
- **Chapter 4: Theoretical & Empirical Result Analysis**
  This chapter evaluates the proposed detection framework through both theoretical analysis and empirical validation. It describes the developed tool's interface and core functionalities in detail, outlines methods for testing sample web applications, and thoroughly analyzes the generated results to validate the detection logic.
- **Chapter 5: Conclusion**
  This section summarizes the critical findings and highlights the project's contributions to web security. It emphasizes the significance of effectively addressing HRS vulnerabilities and underscores the practical benefits offered by the HRS Detector tool.
- **Chapter 6: Future Scope**
  This chapter explores opportunities for future enhancements of the HRS Detector tool, suggesting upgrades in protocol support, refinements in detection logic, improved payload crafting, and greater integration and automation capabilities.
- **Chapter 7: Bibliography**
  This chapter lists all references and resources consulted during the project, ensuring proper attribution and facilitating further study.
- **Chapter 8: Plagiarism Report**
  This final section presents the results of a plagiarism check, confirming the originality and authenticity of the presented work.

2. Theoretical Background & Literature Survey

2.1 Introduction to HTTP Protocol

Hypertext Transfer Protocol (HTTP) is the foundational communication protocol used to transfer data across the World Wide Web. It serves as the primary method by which web browsers, servers, and other web services communicate.

Key Characteristics of HTTP:

- **Stateless Nature:** HTTP does not retain information about previous requests between client (e.g., web browser) and server interactions. Every request is processed as an independent transaction.
- **Request-Response Model:** Communication follows a straightforward request-response sequence, where a client initiates a request, and the server provides a response. This simplicity makes HTTP easy to implement and manage.
- **Layered Architecture:** HTTP typically operates over TCP/IP, leveraging a reliable transport mechanism. This layered structure allows HTTP to integrate seamlessly with other protocols and security standards, such as TLS.

Typical HTTP Communication Process:

1. **Client Sends a Request:** When a user navigates to a webpage, the browser sends an HTTP request to the server, specifying methods like GET or POST.
2. **Server Processes the Request:** The server analyzes the requested resource and prepares the appropriate response, which may include HTML, images, or other media.
3. **Server Returns a Response:** The server sends back a response containing status codes (e.g., 200 OK), headers (such as content type), and the requested content.

1. **Client Renders the Response:** The browser interprets the received data, renders the webpage to the user, and initiates additional HTTP requests if necessary for embedded resources like images, scripts, and styles.

2.1.1 HTTP 1.X

HTTP 1.0 – The Early Web Era

HTTP 1.0, introduced around 1996, was one of the first standard versions of the Hypertext Transfer Protocol. At this stage, the web was still in its infancy, and the focus was on creating a straightforward method for transferring documents over the internet. Although basic by today's standards, HTTP 1.0 laid the foundational ground for the interactive, content-rich web we know now.

Key Features of HTTP 1.0

1. **Simple, Text-Based Communication:**
   HTTP 1.0 requests and responses are communicated as plain text. Both the client and server easily understand this format, making it simple to implement.

2. **Single-Request Connections:**
   Each request and response pair typically results in its own separate TCP connection. After the server sends its response, the connection is closed.

   Implication:

   – This approach was simple but not efficient, as opening and closing a new connection for every resource (like images or stylesheets) introduced latency and load on both clients and servers.

3. **Limited Caching and Headers:** HTTP 1.0 had very basic support for caching through a few headers (like `Expires`), but this was not as well-defined or flexible as in later versions. The headers used were minimal and focused on the essentials (e.g., content type, content length).

4. **Lack of Persistent Connections:** Without the concept of "keep-alive," clients had to re-establish new connections for every new request. This was not ideal for complex web pages that required multiple linked resources.

5. **One Resource per Request:** Each HTTP request in 1.0 was primarily designed to fetch a single resource (e.g., an HTML page or an image). For web pages containing multiple resources, the client had to make multiple requests, each starting and ending its own connection.

Example of an HTTP 1.0 Request and Response

Client Request (simplified):

```
GET /index.html HTTP/1.0
Host: www.example.com
```

Server Response (simplified):

```
HTTP/1.0 200 OK
Content-Type: text/html
Content-Length: 1024

<HTML>
<!-- HTML content of 1024 bytes -->
</HTML>
```

- The client sends a `GET` request for `index.html`.

- The server responds with a status line (`200 OK`) and basic headers, followed by the content.
- After sending the response, the server closes the connection.

Impact and Limitations

- **Slow Page Loads:** For a webpage with multiple images or scripts, the browser had to initiate multiple separate requests, making pages load slowly.
- **Minimal Security Features:** HTTPS existed but was less common, and HTTP 1.0 didn't have sophisticated security features built into the protocol.

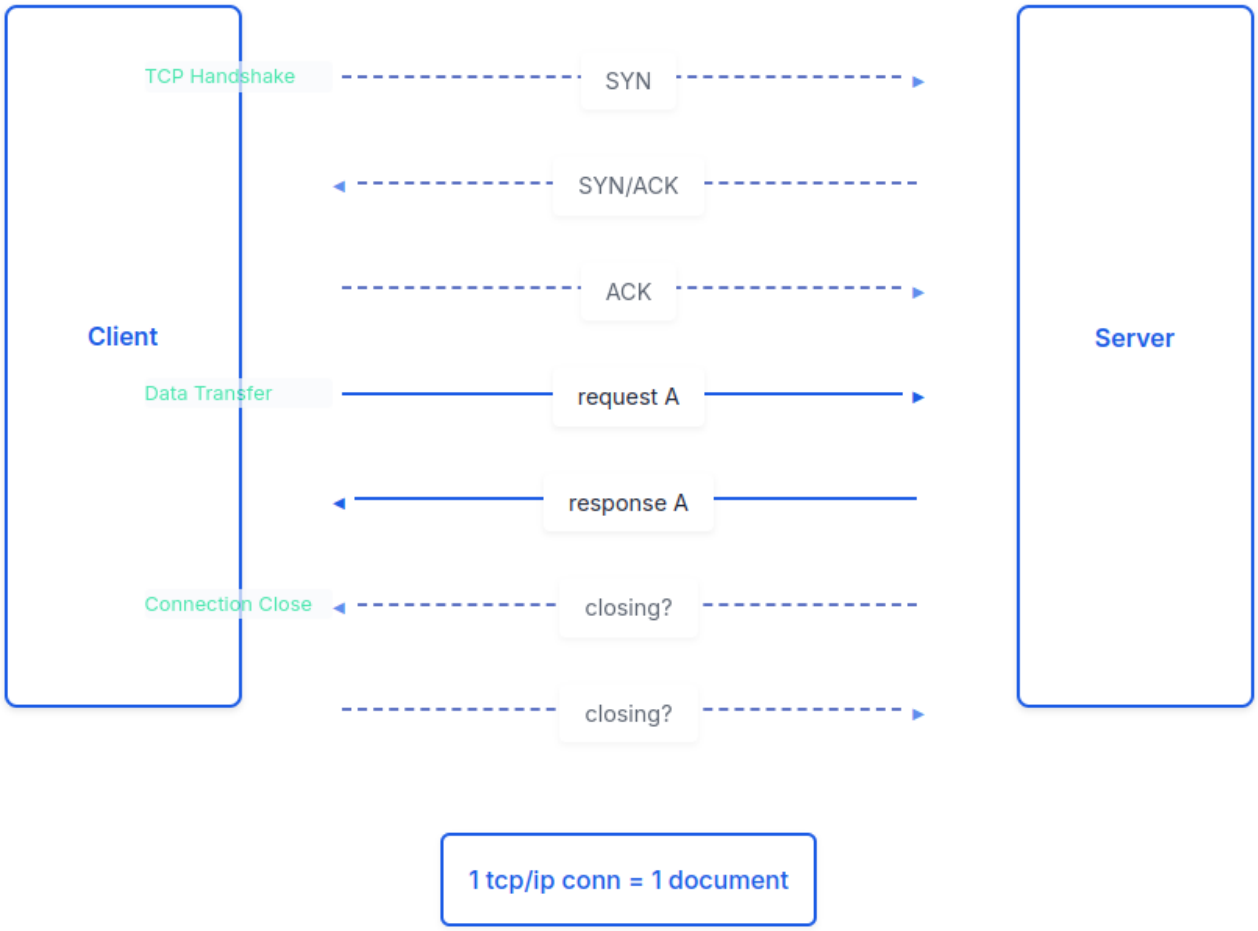


Figure 1 HTTP 1.0

HTTP 1.1 – A More Efficient and Flexible Web Protocol

Released in 1997, **HTTP 1.1** built upon the foundation laid by HTTP 1.0. The goal was to tackle performance and efficiency problems and provide more robust features for modernizing the web experience as sites became richer and more interactive.

Key Improvements Over HTTP 1.0

1. **Persistent Connections (Keep-Alive):**
   With HTTP 1.1, a single TCP connection can be reused for multiple requests. This eliminates the overhead of repeatedly opening and closing connections for each resource and greatly improves loading times, especially on web pages with multiple images, stylesheets, and scripts.

2. **Chunked Transfer Encoding:**
   HTTP 1.1 introduces a way to send data in "chunks" without knowing the entire size of the response at the start. This is useful for dynamically generated content or streaming data, as the server can send data as it becomes available.

3. **Better Caching Mechanisms:**
   The specification enhances caching through more detailed headers like `Cache-Control` and `ETag`. These additions give servers and clients fine-grained control over how resources are stored and reused, reducing unnecessary re-downloads and speeding up page loads.

4. **Content Negotiation:**
   HTTP 1.1 allows clients and servers to "negotiate" the form of content. For example, a browser can say it prefers `gzip`-compressed data or a particular language, and the server can respond accordingly, leading to more efficient and user-friendly delivery of content.

5. **Additional HTTP Methods:**
   While HTTP 1.0 primarily focused on `GET`, `POST`, and `HEAD`, HTTP 1.1 formalized additional methods such as `PUT`, `DELETE`, `OPTIONS`, and `TRACE`. This broadened the range of actions that clients and servers can perform, laying a foundation for more advanced web applications and RESTful APIs.

6. **Improved Status Codes and Headers:**
   HTTP 1.1 refines status codes and introduces new headers that give clients and servers more information. For example, `100 Continue` can help a client confirm that a server is ready to receive a large request body before actually sending it.

Example of an HTTP 1.1 Request and Response

Client Request (simplified):

```
GET /images/logo.png HTTP/1.1
Host: www.example.com
Connection: keep-alive
Accept-Encoding: gzip, deflate
```

Server Response (simplified):

```
HTTP/1.1 200 OK
Content-Type: image/png
Content-Length: 2048
Cache-Control: max-age=3600
```

- **Reused Connection:** The browser and server can continue to use the same TCP connection for subsequent requests on the same page.
- **Cache Control:** The server directs the client to cache the image for an hour (3600 seconds).
- **Content Encoding:** If the image were text-based and supported compression, the server could send a compressed version requested by the client's `Accept-Encoding` header.

Performance and Practical Benefits

- **Faster Page Loads:** Persistent connections mean fewer delays caused by repeatedly setting up and tearing down TCP connections.
- **Reduced Network Overhead:** More efficient use of network resources and less congestion.
- **Richer Interactions:** The ability to handle more request methods and better caching allowed the web to mature into dynamic, application-like experiences.

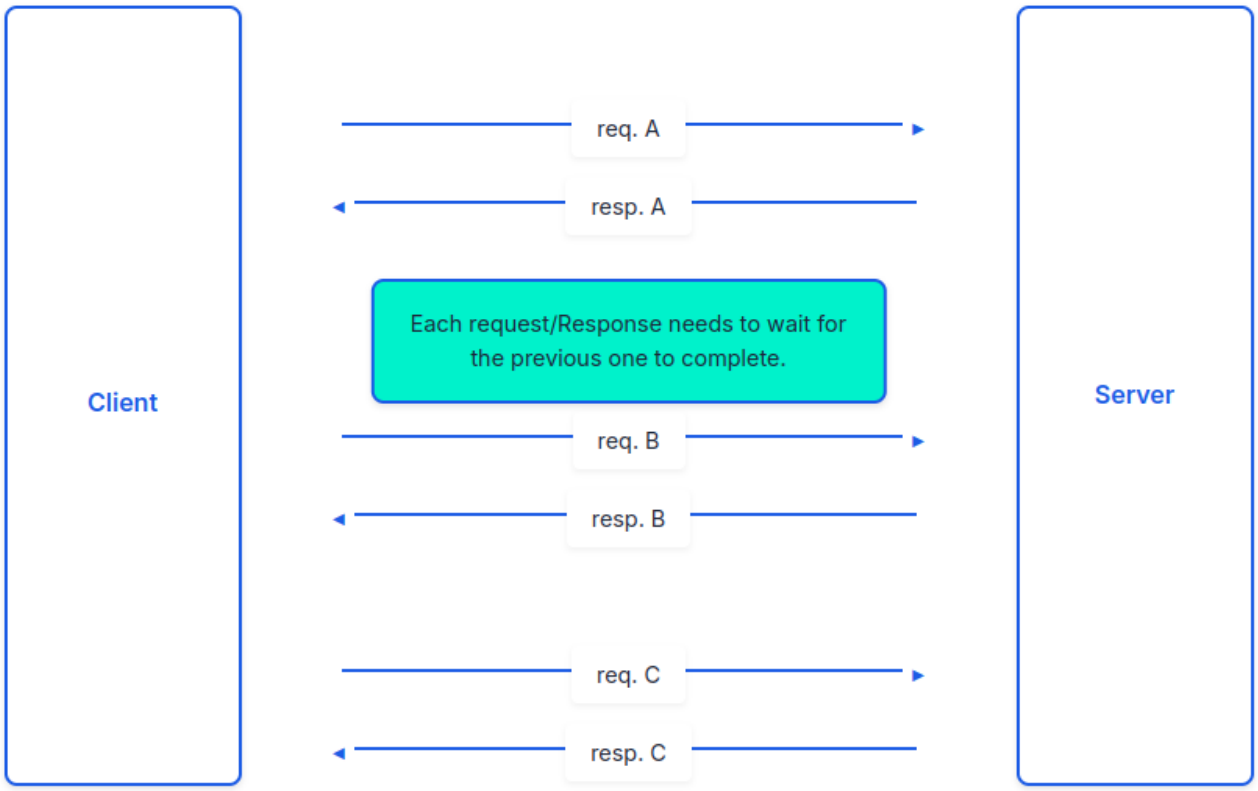


Figure 2 HTTP 1.1 Keep-Alive

HTTP Pipelining

HTTP Pipelining is a feature introduced in HTTP/1.1 that allows a client to send multiple requests over a single persistent connection without waiting for each corresponding response before sending the next request. This approach aims to reduce the time spent establishing new connections and improve overall loading performance.

How HTTP Pipelining Works

1. **Single Connection, Multiple Requests:**
   - Normally, even with persistent connections, a client would send a request and then wait for the server's response before issuing the next one.
   - With pipelining, the client can send several requests back-to-back over the same connection.
2. **Server Responses in Order:**
   - The server processes these requests in the order they were received.
   - Responses are returned in the exact order the requests were sent to avoid confusion.
3. **Performance Benefits:**
   - By sending multiple requests at once (for example, requesting multiple images on a webpage), the client potentially reduces the waiting time between responses.

Limitations and Challenges

- **Head-of-Line Blocking:** If one of the early requests in the pipeline takes a long time to process, it holds up all subsequent responses. This is known as "head-of-line blocking" and can actually degrade performance in certain scenarios.
- **Transition to HTTP/2:** With the arrival of HTTP/2, which multiplexes multiple requests and responses more efficiently, interest in pipelining declined. HTTP/2 effectively solves the head-of-line blocking issue by allowing multiple requests and responses to interleave without waiting in line.

Example Scenario

- **Without Pipelining:**
  The client requests `image1.png`, waits for a response, then requests `image2.png`, waits, and so on. Each request-response cycle adds delay.
- **With Pipelining:**
  The client sends requests for `image1.png`, `image2.png`, and `image3.png` all at once. The server processes them in sequence and returns the responses in order. While this can improve speed, if `image1.png` takes too long, it might delay the others.

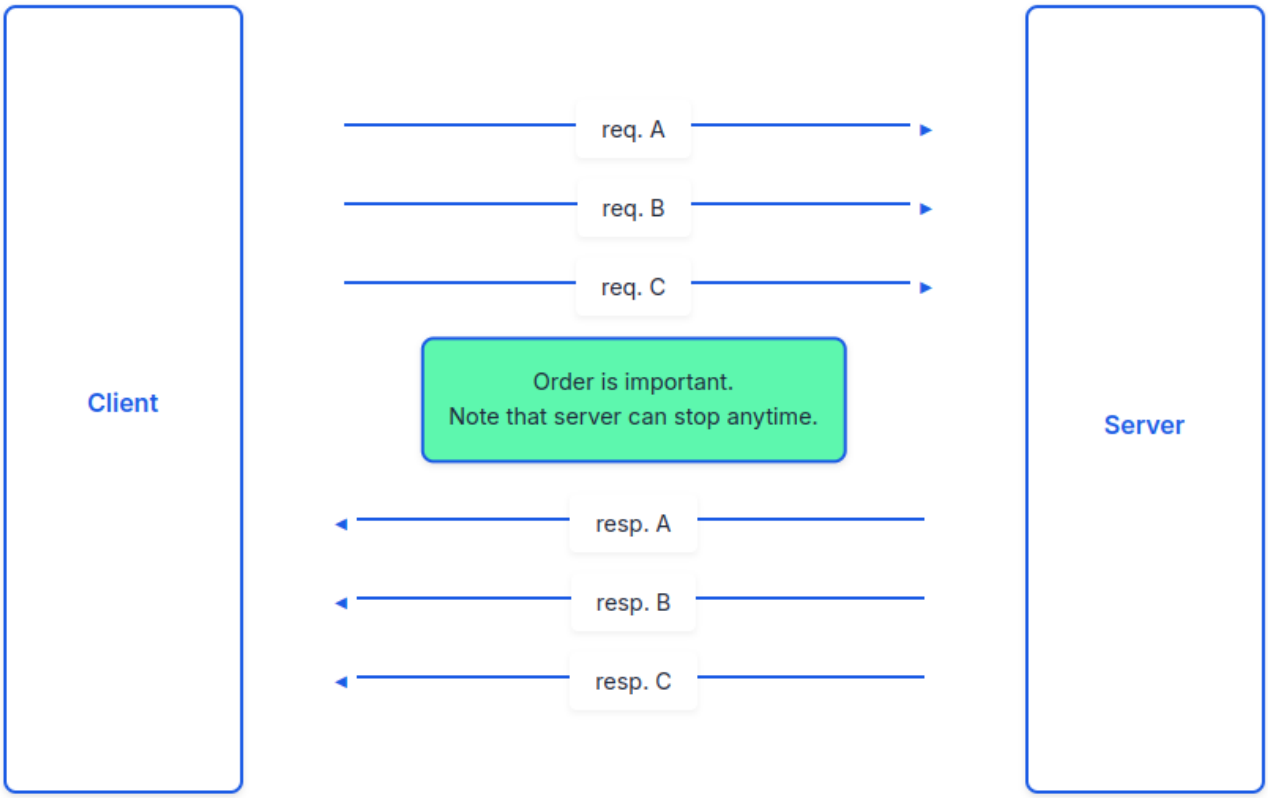


Figure 3 HTTP 1.1 Pipelining

2.1.2 HTTP 2

HTTP/2 (also known as HTTP/2.0) was standardized in 2015 as the next generation of the HTTP protocol. It builds on the fundamentals of HTTP/1.1 but introduces several key enhancements aimed at improving speed, efficiency, and overall user experience when browsing the web.

Key Goals of HTTP/2

- **Faster Loading Times:**
  HTTP/2 focuses on reducing latency, meaning it helps websites load quicker. This is achieved by letting clients and servers exchange data more efficiently.
- **Better Resource Handling:**
  With features like multiplexing and server push, HTTP/2 enables the transfer of multiple resources (e.g., text, images, scripts) more smoothly and with fewer delays.

Major Features of HTTP/2

1. **Binary Framing:**
   - Unlike HTTP/1.1, which uses a text-based format, HTTP/2 communicates data in a binary form.
   - Binary data is simpler for computers to handle, making parsing faster and more efficient. This reduces overhead and can improve performance.
2. **Multiplexing:**
   - Under HTTP/1.1, even with persistent connections, requests often ended up waiting in line, causing delays (head-of-line blocking).
   - HTTP/2 allows multiple requests and responses to be in flight simultaneously over a single connection.
3. **Header Compression:**
   - HTTP/2 compresses request and response headers, which often repeat the same data (like cookies, user agents, and accept-language).
4. **Server Push:**
   - With server push, a server can send resources to the client before the client even requests them.
   - **Example:**
     When the client requests the main HTML page, the server can “push” related stylesheets, scripts, and images ahead of time, so they are ready to be displayed as soon as the page tries to load them.

How HTTP/2 Improves the Web Experience

- **Reduced Latency:**
  Faster data transfer and the ability to send multiple pieces of content simultaneously mean webpages generally load faster, especially on high-latency networks.
- **Less Overhead:**
  Using binary frames and compressed headers helps reduce the amount of unnecessary data traveling over the network.
- **More Efficient Use of Connections:**
  Multiplexing ensures that only one TCP connection is needed per origin. This reduces connection overhead and makes networking smoother.
- **Browser Support:**
  Most modern browsers (Chrome, Firefox, Safari, Edge) and many web servers support HTTP/2, ensuring a wide range of compatibility.
- **Security (TLS):**
  While HTTP/2 does not strictly require HTTPS, in practice, most browsers only support HTTP/2 over encrypted connections (HTTPS). This encourages better security practices across the web.

Understanding Pseudo-Headers in HTTP/2

Pseudo-headers are a special kind of header used by HTTP/2 to convey essential parts of the request that were previously found on the request line in HTTP/1.1 (like the method, path, and scheme). They are always written before the regular headers and start with a colon (`:`).

Typical pseudo-headers include:

- `:method` – The HTTP method (e.g., GET, POST).
- `:scheme` – The scheme of the target URL (e.g., http or https).
- `:authority` – The authority portion of the URL (e.g. www.example.com).
- `:path` – The path of the resource being requested (e.g., /index.html).

These headers must appear at the beginning of the HEADERS frame, in a specific order, and cannot appear more than once.

Conceptual Example of an HTTP/2 Request

Imagine we want to request `https://www.example.com/index.html` using HTTP/2. A simplified representation of the request HEADERS might look like this:

```
:method: GET
:scheme: https
:authority: www.example.com
:path: /index.html

user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
accept-encoding: gzip, deflate, br
accept-language: en-US,en;q=0.5
```

- **Pseudo-headers:**
  - `:method: GET` states we are making a GET request.
  - `:scheme: https` specifies that we are using the HTTPS scheme.
  - `:authority: www.example.com` shows the server we're contacting.
  - `:path: /index.html` is the resource we want from that server.
- **Regular Headers:**
  After the pseudo-headers, we have familiar HTTP headers like `user-agent`, `accept`, `accept-encoding`, and `accept-language`. These work similarly to HTTP/1.1 headers, giving the server more context about what the client can handle and what it prefers.

Key Points:

- The order of pseudo-headers is important.
- No pseudo-header can appear after a regular header.
- These headers are handled and interpreted by the server's HTTP/2 decoder, which then knows how to route and fulfill the request.

2.2 Modern Web Architecture

Frontend and Backend Servers

Overview.

Modern web application architecture typically consists of multiple layers of servers and services. At a high level, you can imagine a chain of request handling that starts with the user's browser and ends at the application's core logic stored on backend servers. Between these endpoints lie various frontend servers or intermediary components. These frontend servers handle tasks like distributing load, caching content, enhancing security, and simplifying how data travels through the system. The core idea is to deliver an efficient, scalable, secure, and maintainable experience for both end-users and developers.

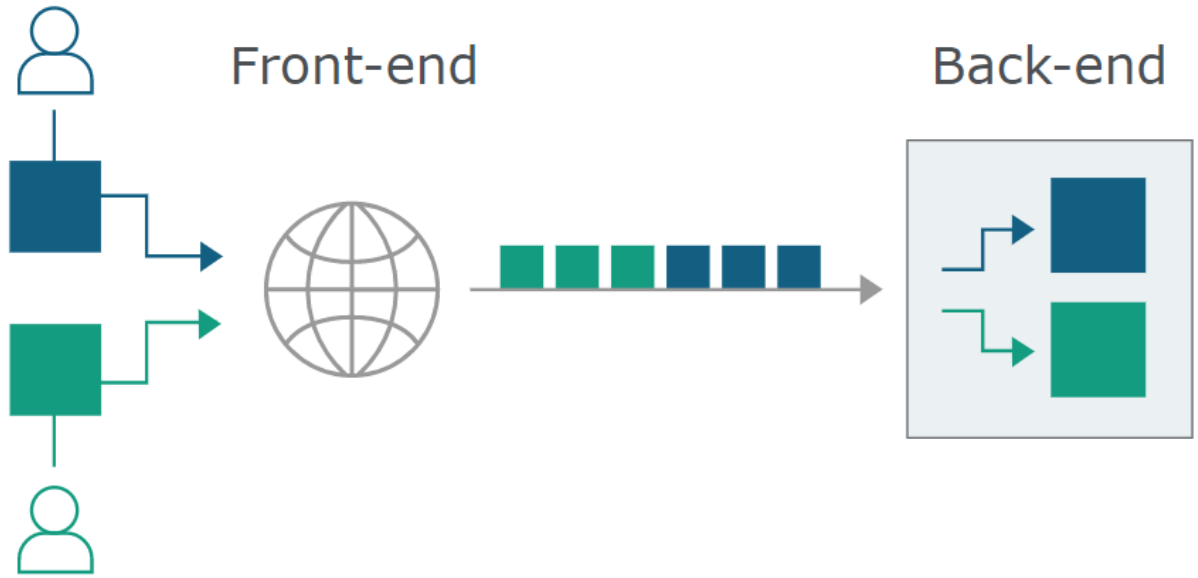


Figure 4 Modern Frontend - Backend Architecture

Key Concepts in a Layered Architecture

1. **Client (User Browser or Application):**
   - This is where a user initiates a request—for example, opening a website link.
   - The client sends an HTTP request to a known web address (e.g., `https://example.com`).
2. **Frontend Servers (Intermediary Layer):**

   Definition: Frontend servers are not just the "client-side code" that runs in the browser. In the context of modern web architecture, the term "frontend servers" refers to the infrastructure components that exist between the user and the final backend servers. These components often include:

   - **Load Balancers:** Distribute incoming traffic across multiple servers to ensure no single server is overwhelmed.
   - **Reverse Proxies:** Act as a gateway, forwarding requests from clients to backend servers, handling SSL termination, and implementing caching or filtering.
   - **Content Delivery Networks (CDNs):** Geographically distributed caches that store static files (like images, scripts, stylesheets) closer to users for faster load times.

Role: Frontend servers help smooth out the interaction between the external world and internal systems. They can:

- Reduce latency through caching and geographic distribution.
- Improve security by filtering malicious traffic before it reaches critical backend servers.
- Enhance reliability by rerouting requests if a backend server is down or under heavy load.

3. **Backend Servers (Core Logic and Data):**

These servers host the application's core functionalities—such as processing business logic, interacting with databases, and running essential services.

Common examples include:

- **Application Servers:** Where the main application logic runs.
- **Database Servers:** Store, retrieve, and manage application data.
- **Microservices Containers:** Independently deployable services that handle specific functions (e.g., user authentication, payment processing).

Backend servers are often hidden behind multiple layers of frontend servers and proxies, making them more secure, stable, and easier to scale.

Example in Practice

- A user access `https://shop.example.com`.
- The request hits a CDN that quickly serves images and static files.
- A load balancer then routes the request for dynamic content to a reverse proxy.
- The reverse proxy checks a WAF for suspicious patterns and then forwards the request to the backend.
- The backend microservice retrieves product details from a database and returns the data.
- The response is sent back through the proxy and load balancer, potentially cached for future requests, and finally returned to the user's browser.

2.3 Understanding HTTP Request Processing

Overview

When discussing HTTP request parsing, we focus on the process at a fundamental level—how bytes arrive at the server, how boundaries between headers and body are recognized, and how the server interprets this information according to HTTP's rules.

The Structure of a Raw HTTP Request

A raw HTTP request generally has the following structure:

```
<method> <request-target> <HTTP-version>
<header-field-1>: <header-value-1>
<header-field-2>: <header-value-2>
...
<header-field-n>: <header-value-n>

[Optional message body follows this blank line]
```

- **Request Line:** The first line contains the HTTP method (e.g., GET, POST), the request target (e.g., `/index.html`), and the protocol version (e.g., `HTTP/1.1`).
- **Headers:** Subsequent lines consist of header fields, each defining metadata about the request. Headers are in `Name: Value` format, one per line.
- **Blank Line:** A blank line (CRLF alone) indicates the end of the header section.
- **Body (Optional):** Depending on the method and headers (e.g., `Content-Length`, `Transfer-Encoding`), an optional message body may follow.

Step-by-Step Parsing of a Raw HTTP Request

1. **Initial Network Read (TCP Stream):**
   - The server receives raw data as a stream of bytes over a TCP connection.
   - HTTP is a stateless, text-based protocol layered on top of TCP. There is no explicit "start" or "stop" marker at the TCP level. Instead, the server must detect message boundaries using the HTTP format rules.
2. **Reading the Request Line:**
   - The parser reads bytes until it encounters a carriage return and line feed (`\r\n`), signifying the end of a line.
   - The first line parsed is the request line (e.g., `GET /index.html HTTP/1.1`).
   - The parser splits this line by spaces:
     - **Method:** Identifies the action (GET, POST, etc.).
     - **Request Target (URI):** The resource the client wants (e.g., `/index.html`).
     - **HTTP Version:** Indicates how the server should interpret the request (e.g., HTTP/1.1).

3. **Reading Headers Line-by-Line:**
   - After the request line, the parser reads subsequent lines until it encounters a blank line (`\r\n\r\n` without preceding characters).
   - Each header line is split into a name-value pair at the first colon (`:`). The server typically normalizes these keys (e.g., case-insensitivity) and may trim whitespace.
   - Important headers for parsing logic include:
     - **Content-Length:** Indicates the exact number of bytes in the message body.
     - **Transfer-Encoding:** May specify chunked encoding, requiring special parsing logic.
     - **Host:** Defines the target host, crucial for virtual hosting and routing.
     - **Connection:** Might specify `keep-alive` or `close`, influencing whether the server keeps the TCP connection open after the response.
4. **Determining Message Length:**
   - Once the headers are all read, the server identifies how to determine the request body length:
     - If `Content-Length` is present, it reads exactly that many bytes after the blank line.
     - If `Transfer-Encoding: chunked` is used, the body is read in variable-sized chunks, each prefixed with a hexadecimal chunk size.
     - If neither is present and the method typically has no body (like GET), then the server expects no message body.
5. **Reading and Parsing the Body (If Present):**
   - If `Content-Length` is provided, the server simply reads the specified number of bytes as the request body.
   - If `Transfer-Encoding: chunked` is in effect:
     - The server reads chunk sizes line-by-line.
     - Each chunk size line is followed by that many bytes of data, and then another `\r\n`.
     - A zero-size chunk followed by \r\n\r\n indicates the end of the message body.
   - If multiple encodings or compressions (e.g., `Content-Encoding: gzip`) are specified, the server may need to decode the data before processing.

2.4 HTTP Request Smuggling Vulnerability

HTTP Request Smuggling (HRS) is a critical web security vulnerability that arises from inconsistencies in how different web systems—such as proxies, load balancers, and web servers—interpret and process HTTP requests. These inconsistencies enable attackers to insert ("smuggle") malicious HTTP requests through intermediate systems, bypassing security controls and potentially triggering unauthorized actions.

The vulnerability specifically exploits ambiguities in parsing HTTP headers, primarily focusing on discrepancies between the interpretation of the **Content-Length** and **Transfer-Encoding** headers. By manipulating these headers, attackers can control how requests are processed, leading to unintended behaviours and security breaches.

2.4.1 Definition and Concepts

HTTP Request Smuggling is a technique used by attackers to exploit differences in the way HTTP requests are processed by interconnected servers in a communication chain. Typically, a request passes through multiple systems like load balancers, proxies, and web servers before reaching its final destination. If these systems interpret the same request differently, it can lead to vulnerabilities.

- A reverse proxy might use the `Content-Length` header to determine the end of a request.
- A backend server might instead prioritize the `Transfer-Encoding` header for the same purpose.

This discrepancy allows an attacker to craft a single HTTP request that one system interprets as single valid request, while another sees it as two distinct requests. This manipulation can result in:

- **Request concatenation**: Malicious request gets appended to legitimate requests.
- **Socket poisoning**: The attacker gains control of a communication channel, affecting subsequent requests.

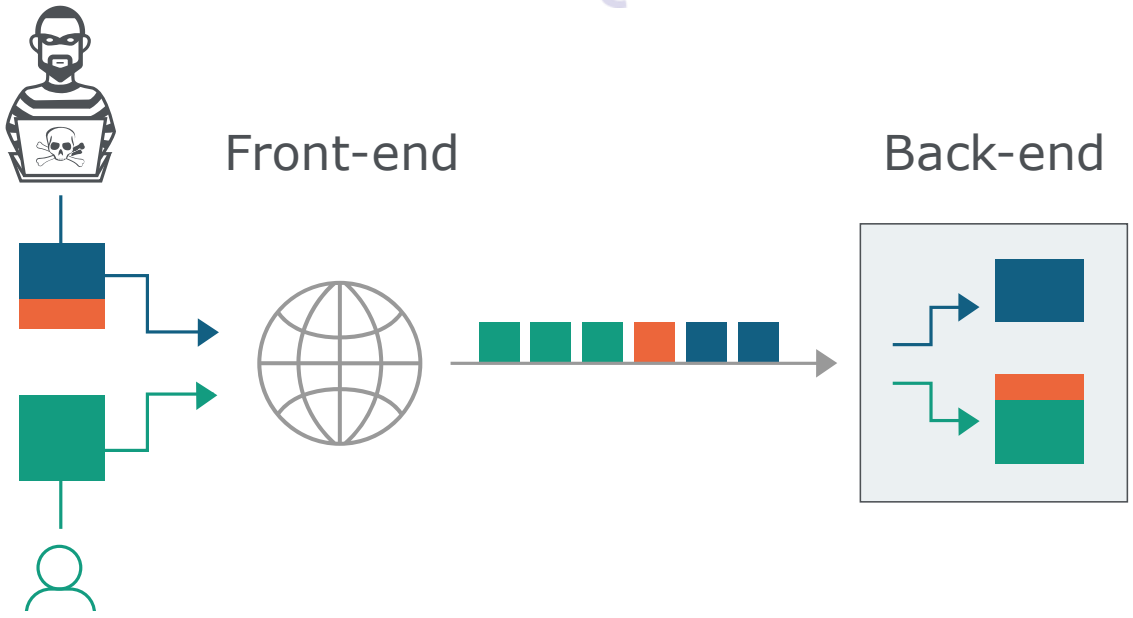


Figure 5 HRS Illustration

Key Concepts in HTTP Request Smuggling:

1. **HTTP Protocol Mechanisms**:
   - The attack primarily exploits the HTTP/1.1 protocol, which introduced features like persistent connections and pipelining. These features allow multiple requests to be sent over a single connection, increasing the chance of parsing inconsistencies.
2. **Parsing Discrepancies**:
   - HTTP devices like proxies and servers often implement their parsers independently, leading to differences in how headers and request bodies are processed.
   - A poorly implemented parser might fail to reject ambiguous or malformed requests, which attackers can exploit.
3. **Multi-System Architectures**:
   - In modern web applications, an HTTP request often traverses several systems (e.g., Content Delivery Networks, load balancers, proxies) before reaching the backend server.
   - The diversity in systems increases the likelihood of mismatched interpretations of the same request.
4. **Types of Headers Involved**:
   - **Content-Length**: Indicates the length of the request body in bytes.
   - **Transfer-Encoding**: Specifies the method used to encode the body, often `chunked` for streamed data.
   - If both headers are present, ambiguities arise about which one to prioritize, depending on the server.

Impact of HRS:

HRS can have severe consequences for web applications, including:

- **Web Cache Poisoning**: Altering cached data to serve malicious content to users.
- **Session Hijacking**: Stealing session cookies or tokens to impersonate legitimate users.
- **Firewall Bypassing**: Sending malicious payloads undetected by circumventing security layer.

2.4.2 Types of HTTP Request Smuggling Vulnerability

HTTP Request Smuggling vulnerabilities are categorized based on the parsing discrepancies exploited between HTTP systems. These include:

- **CL.TE (Content-Length . Transfer-Encoding)**: Occurs when the front-end server uses the `Content-Length` header, while the back-end relies on the `Transfer-Encoding` header to determine request boundaries.
- **TE.CL (Transfer-Encoding . Content-Length)**: The inverse of CL.TE, where the front-end interprets the `Transfer-Encoding` header, and the back-end prioritizes the `Content-Length` header.
- **CL.CL (Dual Content-Length)**: Happens when two `Content-Length` headers are present, and the front-end and back-end prioritize different header, leading to misinterpretation. (not common)
- **TE.TE (Dual Transfer-Encoding)**: Exploits discrepancies in how multiple `Transfer-Encoding` headers are interpreted by front-end and back-end systems. (not common)
- **CL.0**: Exploits parsing anomalies by introducing `Content-Length: SomeLength`, tricking frontend server into treating the request with the body length as given in CL header, while backend server does not consider CL value thus assumes it to be Zero.
- **H2.CL (HTTP/2 . Content-Length)**: Occurs during HTTP/2-to-HTTP/1 conversion (HTTP Downgrading), where the front-end interprets HTTP/2 frames, but the back-end processes the request using a `Content-Length` header, leading to desynchronization.
- **H2.TE (HTTP/2 . Transfer-Encoding)**: Similar to H2.CL but leverages `Transfer-Encoding` discrepancies during HTTP/2-to-HTTP/1 conversion.

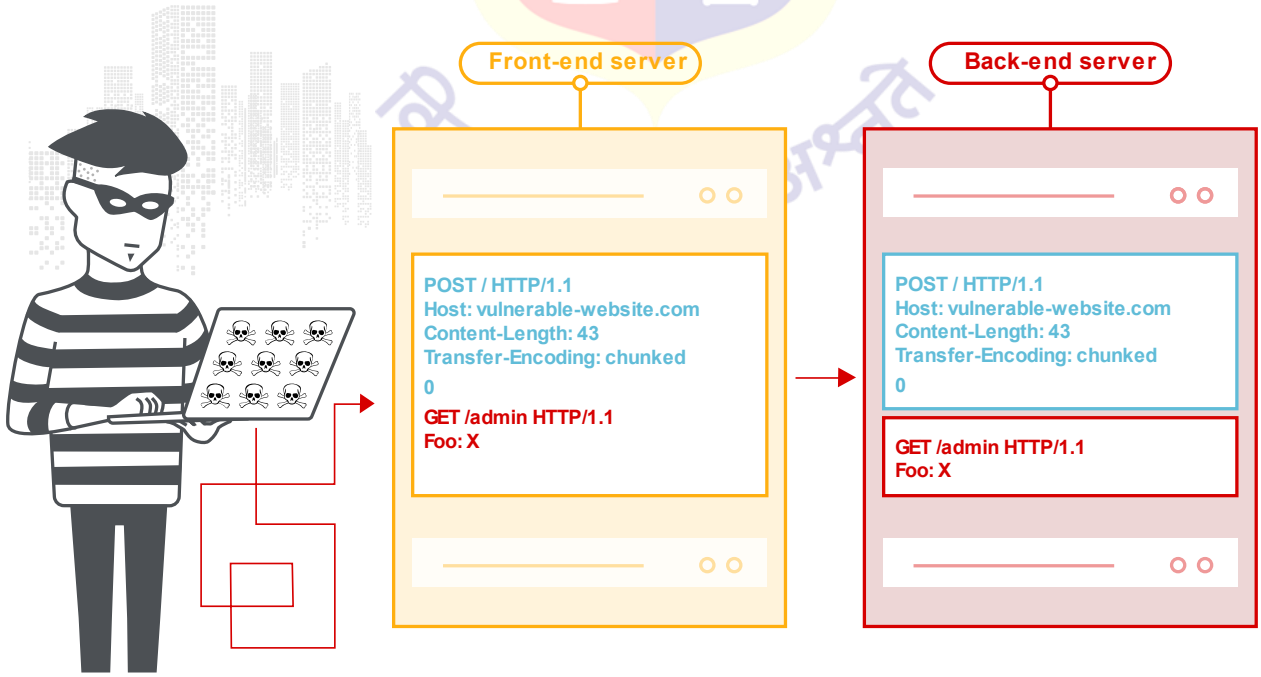


Figure 6 HRS Raw Request

2.4.2.1 CL.TE

CL.TE vulnerabilities occur when a front-end server interprets an HTTP request using the `Content-Length` header to determine the request's boundaries, while the back-end server relies on the `Transfer-Encoding` header. This discrepancy allows an attacker to craft a malicious HTTP request that is interpreted differently by the two servers.

The front-end server sends what it believes to be a single request to the back-end, but the back-end processes additional unintended content as second request i.e. prefix. This leads to desynchronization between the two systems.

How It Works

1. **Front-End Server Behaviour**: The front-end server uses the `Content-Length` header to calculate where the request body ends. It then forwards that request to back-end server with the length of the body calculated using CL.
2. **Back-End Server Behaviour**: The back-end server prioritizes the `Transfer-Encoding` header, interpreting the request body differently. The backend considers it as two different HTTP request, one request is complete and backend waits for another request to complete.
3. **Outcome**: This desynchronization allows the attacker to "smuggle" a second malicious request through the system. This request can be used as prefix for other request that will be subsequently made by victim user or the attacker itself makes this request to exploit the vulnerable system.

>>>> "\r\n" represents CRLF.

>>>> "X" has poisoned backend server's socket, and it will be added as prefix to the next request.

```
POST /vulnerable HTTP/1.1\r\n
Host: example.com\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 6\r\n
\r\n
0\r\n
\r\n
X
```

Figure 7 CL.TE Type HRS

Explanation:

1. **Front-End Server**:
   - Reads the `Content-Length: 6`.
   - Treats the body as 6 bytes: `0\r\n\r\nX`.
   - It takes this request as single HTTP request and forwards this request to backend as it is.
2. **Back-End Server**:
   - Prioritizes `Transfer-Encoding: chunked`.
   - Interprets `0\r\n\r\n` as the end of the chunked request.
   - Treats the remaining Byte (`X`) as start of the request line for new HTTP request.

Using this attacker can inject start of new request in place of "X" as "GET /delete_account HTTP/1.1", And the next Victim request will get concatenated with this and will delete their account.
Most of the time impact of HRS depends upon the existing functionality of the web application.

2.4.2.2 TE.CL

The **TE.CL vulnerability** arises when the front-end server uses the `Transfer-Encoding` header (`chunked`) to parse the request body, while the back-end server relies on the `Content-Length` header. This mismatch allows an attacker to craft a malicious HTTP request that is parsed differently by the two servers, enabling the injection of a smuggled request.

How the Attack Works

1. **Front-End Server Behavior**:
   - The front-end server interprets the body using the `Transfer-Encoding: chunked` header.
   - When in the request body, it finds "0\r\n\r\n" then front-end server thinks the request is complete and forwards that request to the backend server.
2. **Back-End Server Behavior**:
   - The back-end server relies on the `Content-Length` header to determine length of the request body.
   - It consumes the part of the body using CL and remaining part becomes start of the new HTTP request for back-end server.
3. **Outcome**:
   - The attacker causes a desynchronization between the front-end and back-end servers, allowing smuggling of an unauthorized or malicious request.

Here's a crafted example exploiting the TE.CL vulnerability:

```
POST /vulnerable HTTP/1.1\r\n
Host: example.com\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 4\r\n
\r\n
3d\r\n
GET /admin HTTP/1.1\r\n
Host: localhost\r\n
Content-Length:10\r\n

x=\r\n
0\r\n
\r\n
```

Figure 8 TE.CL HRS

Explanation:

- **Front-End Interpretation**:
  - The front-end processes the body as chunked encoding.
  - It considers the body of the request as shown at right.
  - It then forwards the request with this body to the backend.

```
3d\r\n
GET /admin HTTP/1.1
Host: localhost
Content-Length:10

x=
0
\r\n
\r\n
```

- **Back-End Interpretation**:
  - The back-end ignores the `Transfer-Encoding` header and uses the `Content-Length: 4`.
  - So, backend reads 3d/r/n as Four Byte and consumes this request.

Remaining part of the request body starting with "GET /admin HTTP/1.1 ….." is considered as next request by the backend server.

2.4.2.3 CL.CL

The **CL.CL vulnerability** occurs when an HTTP request contains multiple `Content-Length` headers, and the front-end and back-end servers prioritize different values. This discrepancy can lead to inconsistencies in how the request is processed, allowing attackers to inject or manipulate requests. This class of vulnerability is not common these days.

How the Attack Works

1. **Front-End Server Behavior**:
   - The front-end server uses one `Content-Length` header to determine the body size.
   - It forwards the remaining content based on its interpretation.
2. **Back-End Server Behavior**:
   - The back-end server may prioritize a different `Content-Length` header value and interpret the request differently.
   - This leads to desynchronization, allowing the attacker to inject a malicious payload.
3. **Result**:
   - The inconsistency between the two servers enables the attacker to "smuggle" additional data, causing unintended actions or security breaches.

Here's a crafted HTTP request exploiting the CL.CL vulnerability:

```
POST /vulnerable HTTP/1.1\r\n
Host: example.com\r\n
Content-Length: 0\r\n
Content-Length: 1\r\n
\r\n
X
```

Figure 9 HRS Type CL.CL

- **Front-End Interpretation**:
  - The front-end server prioritizes the first `Content-Length: 1`.

- It reads the body as `X` (1 bytes) and forwards to the backend server for processing.

- **Back-End Interpretation**:
  - The back-end server prioritizes the second `Content-Length: 0`.
  - So, backend thinks that server do not have body since CL is 0. The next byte “X” is treated as start of the new request.

2.4.2.4 TE.TE

This vulnerability may exist theoretically but I have not seen any practical implementation of it.

2.4.2.5 CL.0

CL.0 is a specific type of HTTP request smuggling vulnerability. It occurs when there is a mismatch in how the front-end and back-end servers interpret the `Content-Length` header. In other words, it happens when the back-end server effectively ignores the `Content-Length` value and treats the request as if it has no body, while the front-end still respects the `Content-Length`. This discrepancy allows an attacker to "smuggle" hidden or malicious requests into the communication stream.

Key Characteristics of CL.0 Vulnerabilities:

- **No Strange Headers Needed:** Unlike some other request smuggling techniques, a CL.0 attack often doesn’t require malformed or suspicious headers. The attacker uses normal `Content-Length` headers and normal HTTP requests.
- **Backend Assumption:** The back-end server assumes that requests have no body. This often happens for endpoints that only expect simple GET requests or static file requests, so they never consider the possibility of handling a request body.
- **Front-End Behavior:** The front-end (like a load balancer or proxy) will still obey the `Content-Length` header and send the request body accordingly, not realizing the back-end is ignoring it.

Example Scenario:

1. The front-end receives a POST request with a `Content-Length` of 50 bytes.
2. The front-end sends the headers and the subsequent 50 bytes to the back-end.
3. The back-end, due to its configuration or expectation, ignores the `Content-Length` and treats the request as complete immediately after the headers.
4. The extra 50 bytes that the front-end sent are now waiting in the socket buffer and get interpreted by the back-end as a completely new request.

CL.0 Detection Table:

Table 1 CL.0 Detection Table

Step	Action	Expected Behavior (No Vulnerability)	Behavior if Vulnerable
1	Send a POST request with a normal `Content-Length` and embedded request-like data in its body.	The back-end should process the POST and ignore the extra data as just the request body.	The back-end treats the embedded request-like data as a new request.
2	Send a follow-up request, such as a simple GET, on the same connection.	The follow-up GET should return a normal response (e.g., a 200 OK with the expected resource).	The follow-up GET might return an unexpected response (e.g., a 404), showing it was influenced by the previously smuggled data.

2.4.2.6 H2.CL

H2.CL is a type of HTTP request smuggling attack that occurs when a front-end server accepts HTTP/2 requests from clients, converts them into HTTP/1.1 before passing them to a back-end server, and fails to properly handle the `Content-Length` header. In simpler terms, it's a situation where the front-end and back-end servers disagree on how long the request is, allowing an attacker to sneak in extra, unauthorized requests.

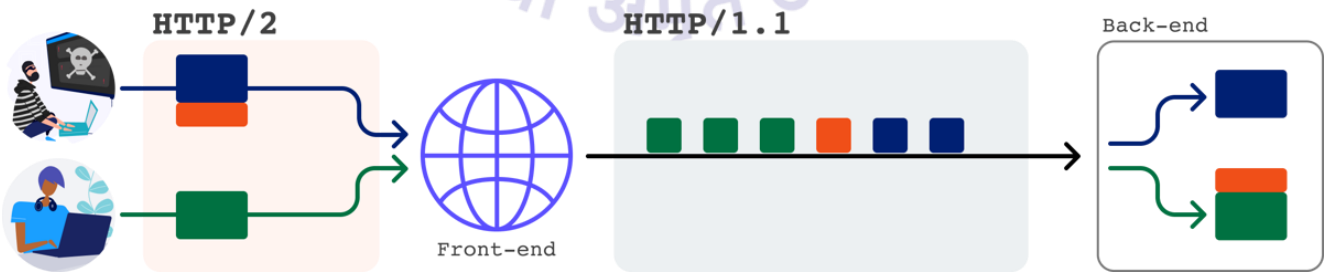


Figure 10 HTTP2 to HTTP/1.1 Downgrade

1. **HTTP/2 to HTTP/1.1 Downgrading:**
   - Many modern web servers or load balancers accept HTTP/2 requests from the client but still use HTTP/1.1 when communicating with the back-end servers.
   - When the front-end downgrades the request, it needs to accurately represent the original HTTP/2 message in HTTP/1.1 form.

2. **Content-Length Mismatch:**

   – HTTP/2 inherently knows the length of its message body by using data frames, so `Content-Length` isn't strictly necessary.

   – However, the HTTP/2 specification says it's allowed to include a `Content-Length` header (it just must be correct if present).

   – If the `Content-Length` is incorrect or ignored by the front-end but honoured by the back-end, it can lead to desynchronization.

   – The back-end will use the provided `Content-Length` to determine where the request ends, while the front-end just uses the HTTP/2 data frames.

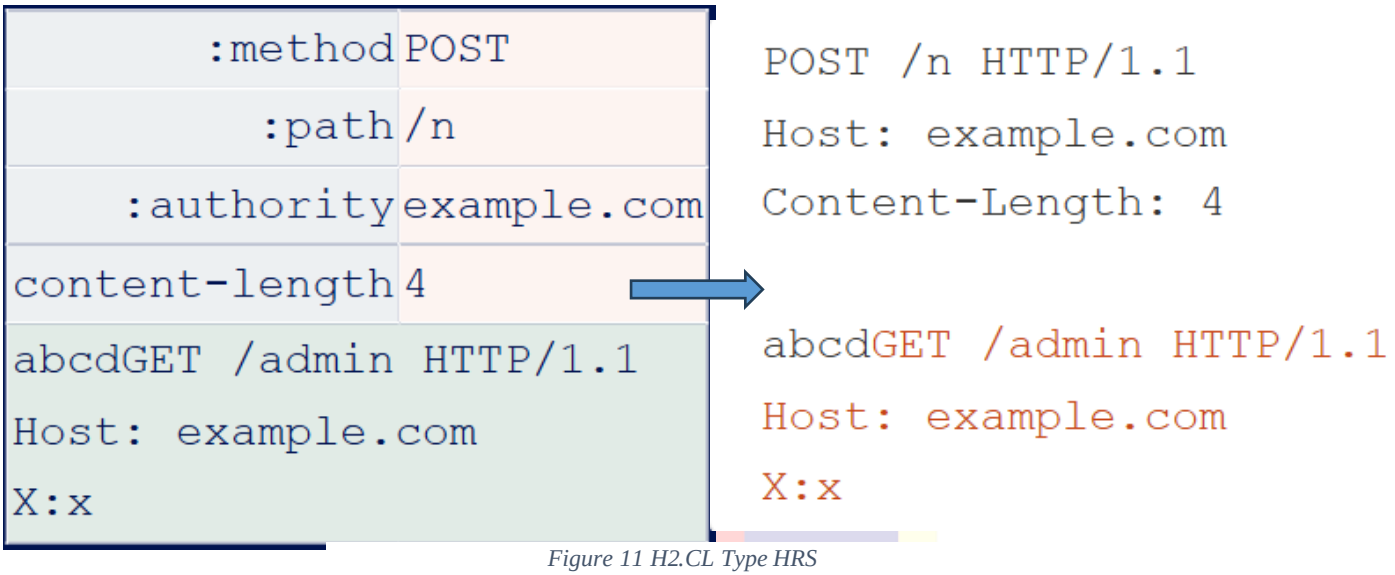


Figure 11 H2.CL Type HRS

- **Injecting Extra Requests:**
  By providing a misleading `Content-Length`, an attacker can cause the back-end to stop reading the original request too early. The extra data that the front-end thought belonged to the original request's body is then interpreted by the back-end as a completely new request.

- **Example Scenario:**

  – The attacker sends a POST request with `Content-Length: 4` but actually includes more data.

  – The front-end reads the whole message (using HTTP/2's built-in framing) and passes it to the back-end.

  – The back-end relies on the `Content-Length: 4`, processes the first four bytes as the request body, and thinks the request is done.

  – The leftover data (beyond those 4 bytes) is now considered a fresh, independent request by the back-end.

2.4.2.7 H2.TE

H2.TE is a request smuggling scenario that arises when a front-end server communicates with clients using HTTP/2 but then forwards requests to the back-end using HTTP/1.1. During this downgrade, if the front-end incorrectly allows or fails to sanitize the `Transfer-Encoding` header, it can lead to a desynchronization between how the front-end and back-end determine where one request ends and the next begins.

1. **HTTP/2 to HTTP/1.1 Downgrading:**
   - The front-end takes an HTTP/2 request and converts it to HTTP/1.1 for the back-end.
   - In HTTP/1.1, `Transfer-Encoding: chunked` is one way to specify the message length.

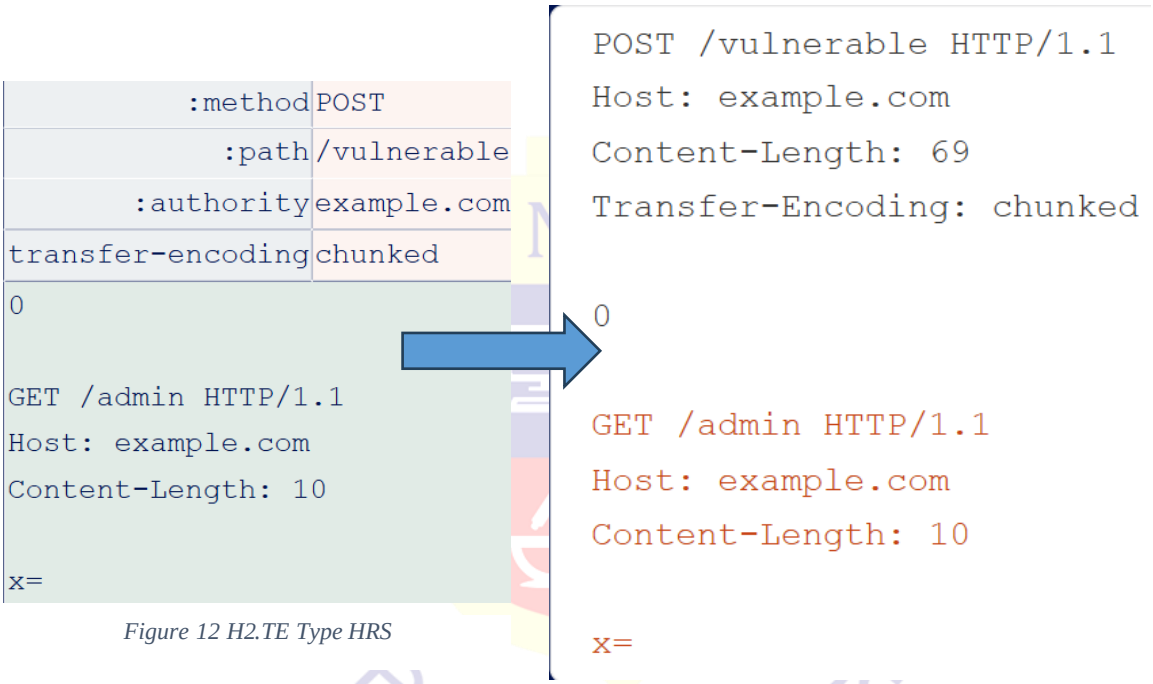


Figure 12 H2.TE Type HRS

2. **Allowed but Unchecked `Transfer-Encoding`:**
   - Under normal circumstances, HTTP/2 does not rely on `Transfer-Encoding` headers since it uses frames to define message lengths.
   - If the front-end doesn't remove or reject `Transfer-Encoding` when converting back to HTTP/1.1, the back-end might parse the incoming request differently.
   - This can trick the back-end into stopping early or late, leading to some parts of the request being misinterpreted as a separate, new request

Imagine the attacker sends a carefully crafted HTTP/2 request:

- It includes a hidden `Transfer-Encoding: chunked` header that the front-end doesn't block.
- When the front-end converts the request to HTTP/1.1, the back-end sees a chunked message.
- Because the back-end trusts `Transfer-Encoding`, it stops reading the request body at the wrong point, treating the rest as a new, separate request controlled by the attacker.

2.4.3 Impact and Exploitation

1. Web Cache Poisoning

- **Description**:
  - Attackers manipulate server responses to poison cache entries with malicious content.
  - Users accessing cached resources receive malicious data instead of legitimate content.
- **Example**:
  - An attacker smuggles a request to cache a malicious file:

```
POST / HTTP/1.1
Host: vulnerable.com
Content-Length: 61
Transfer-Encoding: chunked

0

GET /malicious.js HTTP/1.1
Host: attacker.com
X: x
```

The back-end caches the response to `GET /some-javascript.js(Assumed next request to this endpoint)`, with the content of "malicious.js" from attacter.com server, serving it to future users.

2. Session Hijacking

- **Description**:

Attackers inject requests to steal session cookies or tokens, allowing them to impersonate legitimate users.

- **Example**:

When the request to the vulnerable server is made after attacker has successfully smuggled the prefix request, then the raw http request from the victim is added to attacker's profile page as description and attacker can retrieve session cookies of other sensitive data from it.

```
POST /vulnerable HTTP/1.1
Host: vulnerable.com
Transfer-Encoding: chunked
Content-Length: 4

xx
GET /add_profile_desc HTTP/1.1
Host: vulnerable.com
Content-Length: 300
Cookie: session: attacker_session_cookie;

profile_desc=someinfo
0
```

3. Firewall and Security Control Bypass / Authentication Bypass

- **Description**:
  - Malicious payloads bypass front-end security mechanisms, such as firewalls or Web Application Firewalls (WAF).
- **Example**:
  - A crafted request to bypass input validation:

```
POST /vulnerable HTTP/1.1
Host: vulnerable.com
Content-Length: 46
Transfer-Encoding: chunked

0

DELETE /admin/internal_db_task HTTP/1.1
X:
```

The back-end processes the `DELETE /admin/internal_db_task` command, bypassing the front-end firewall.

2.4.4 Literature Review

Table 2 Literature Review

Serial No.	Paper Name	Author(s)	Published Year	Abstract
1	HTTP Request Smuggling (Watchfire)	Chaim Linhart, Amit Klein, Ronen Heled, Steve Orrin (Watchfire)	2005	Introduced the concept of HTTP request smuggling, demonstrating how discrepancies in interpreting HTTP requests between front-end and back-end servers can allow attackers to inject hidden requests, manipulate responses, and bypass security filters.
2	Hiding Wookiees in HTTP	regilero (Nicolas Grégoire)	2016	Explores advanced techniques for concealing malicious HTTP requests by leveraging subtle differences in how servers parse headers and handle request lengths, thereby enabling stealthy attacks that bypass intrusion detection and filtering.
3	HTTP Request Smuggling Reborn	James Kettle (PortSwigger)	2019	Revitalizes the study of HTTP request smuggling by showcasing newly discovered attack vectors, improved payloads, and effective methods of exploiting modern infrastructures and content delivery networks, despite prior mitigations.
4	HTTP Smuggling via Higher HTTP Version	Emil Lerner and James Kettle	2020	Highlights how leveraging differences between HTTP/1.1, HTTP/2, or newer protocols can create smuggling opportunities. Front-end and back-end servers interpreting protocol versions differently can lead to desynchronized requests and potential security-bypasses.

5	Request Smuggling Via HTTP/2 Cleartext [h2c]	Jake Miller	2020	Demonstrates how attackers can exploit HTTP/2 cleartext (h2c) connections to perform request smuggling attacks. Bypassing front-end security controls.
6	Browser Powered Desync aka Client-Side Desync	James Kettle (PortSwigger)	2021	Presents a new class of attacks wherein the victim's own browser becomes a vector for HTTP request desynchronization. This type of request smuggling does not require even malformed request and then be make via browser and they don't require front-end server too.

3. The Proposed Model and Implementation Methodology

3.1 The Problem Statement

In today's interconnected, service-oriented web landscape, modern applications frequently depend on complex chains of intermediary components—including load balancers, reverse proxies, and caching servers—positioned between clients and backend servers. While these intermediaries significantly enhance performance, scalability, and reliability, they also introduce potential parsing inconsistencies and misinterpretations of HTTP requests. Such discrepancies can be exploited through **HTTP Request Smuggling (HRS)** vulnerabilities, where attackers carefully craft requests to manipulate how different systems interpret and handle incoming data.

This vulnerability is particularly dangerous as it allows attackers to bypass established security controls, tamper with legitimate user requests, hijack active user sessions, or even access sensitive data. The subtle nature and inherent complexity of HRS vulnerabilities make them notoriously difficult to identify and analyze using conventional penetration testing methods. Current detection approaches typically involve tedious manual effort and lack intuitive, user-friendly interfaces. This manual complexity often leads to inefficiencies, human errors, and incomplete vulnerability assessments, posing a heightened challenge for both experienced security experts and less seasoned testers. Consequently, organizational risks related to these vulnerabilities continue to grow.

To effectively address this critical security concern, there is an urgent need for a specialized, automated, and easy-to-use solution for the detection, analysis, and mitigation of HTTP Request Smuggling vulnerabilities. Such a tool would equip security practitioners and penetration testers with the ability to perform thorough assessments efficiently, reduce the risk of human error, and strengthen overall web application security within increasingly complex and dynamic threat environments.

3.1.1 High Level workflow

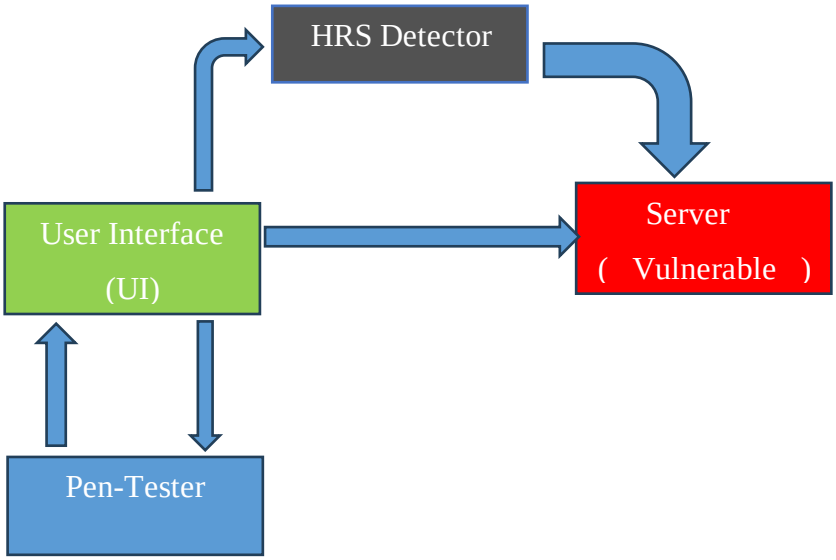


Figure 13 High Level Tool Workflow

1. **Pen-Tester:**

   The pen-tester (penetration tester) is the security professional or analyst responsible for identifying, validating, and exploiting potential vulnerabilities in target applications and servers. Their role involves using various tools and techniques to simulate real-world attacks. In the context of HTTP Request Smuggling, the pen-tester wants to efficiently identify weak points in the web infrastructure and determine if the target servers are susceptible to this class of attacks.

2. **User Interface (UI):**

   The User Interface acts as the primary interaction layer between the pen-tester and the detection tool. Instead of relying on complex command-line operations or intricate configuration files, the UI offers a more intuitive, graphical environment for the tester. With the UI, a pen-tester can:

   – Enter the target URLs, endpoints, or host details to be examined.

   – Adjust scanning parameters, such as timeouts or detection patterns.

   – Initiate the scanning and testing process by sending requests through the tool.

   – View and interpret the results more easily, thanks to structured displays of response data, vulnerability indicators, and actionable insights.

3. **HRS Detector :**
   The "HRS Detector" is the core tool or engine responsible for performing the HTTP Request Smuggling analysis. This component:

   – Interprets input from the UI and constructs HTTP requests designed to test for inconsistencies in the way different servers and intermediaries parse these requests.

   – Sends carefully crafted requests to the target server while observing how responses are handled, looking for signs of request misalignment, truncated responses, or unexpected behaviours.

   – Utilizes logic, heuristics, or known detection patterns derived from the HTTP Request Smuggling methodology to identify and confirm the existence of vulnerabilities.

   Essentially, this module is the "brains" of the operation. By automating the probing and detection process, it streamlines what would otherwise be a tedious and error-prone manual task.

4. **Server (Vulnerable):**
   The server component in the diagram represents the target application's endpoint. This might be:

   – A web server, load balancer, reverse proxy, caching layer, or a combination of these behind a domain.

   – Potentially misconfigured or relying on inconsistent request parsing logic that makes it susceptible to HTTP Request Smuggling.

   As the HRS Detector sends crafted requests, the server's responses and behaviour are closely monitored. If the server (and its associated intermediaries) fails to handle these specially formed requests correctly, it may indicate a vulnerability that allows attackers to manipulate request boundaries, inject malicious payloads, bypass security controls, or intercept and alter responses intended for other users.

Information Flow Explanation:

- The **Pen-Tester** interacts with the **User Interface**, selecting targets and starting the testing process.
- The **UI** passes these instructions and configurations to the **HRS Detector** (the backend logic).
- The **HRS Detector** sends crafted HTTP requests to the **Server** to detect inconsistencies in request handling.
- Based on the server's responses, the **HRS Detector** analyses and forwards findings back to the **UI**, where the pen-tester can review them and take appropriate steps.
- By examining the test outcomes, the pen-tester can confirm the presence of vulnerabilities, better understand the security posture of the server, and plan mitigation strategies.

3.1.2 HRS Detector workflow for CL.TE and TE.CL

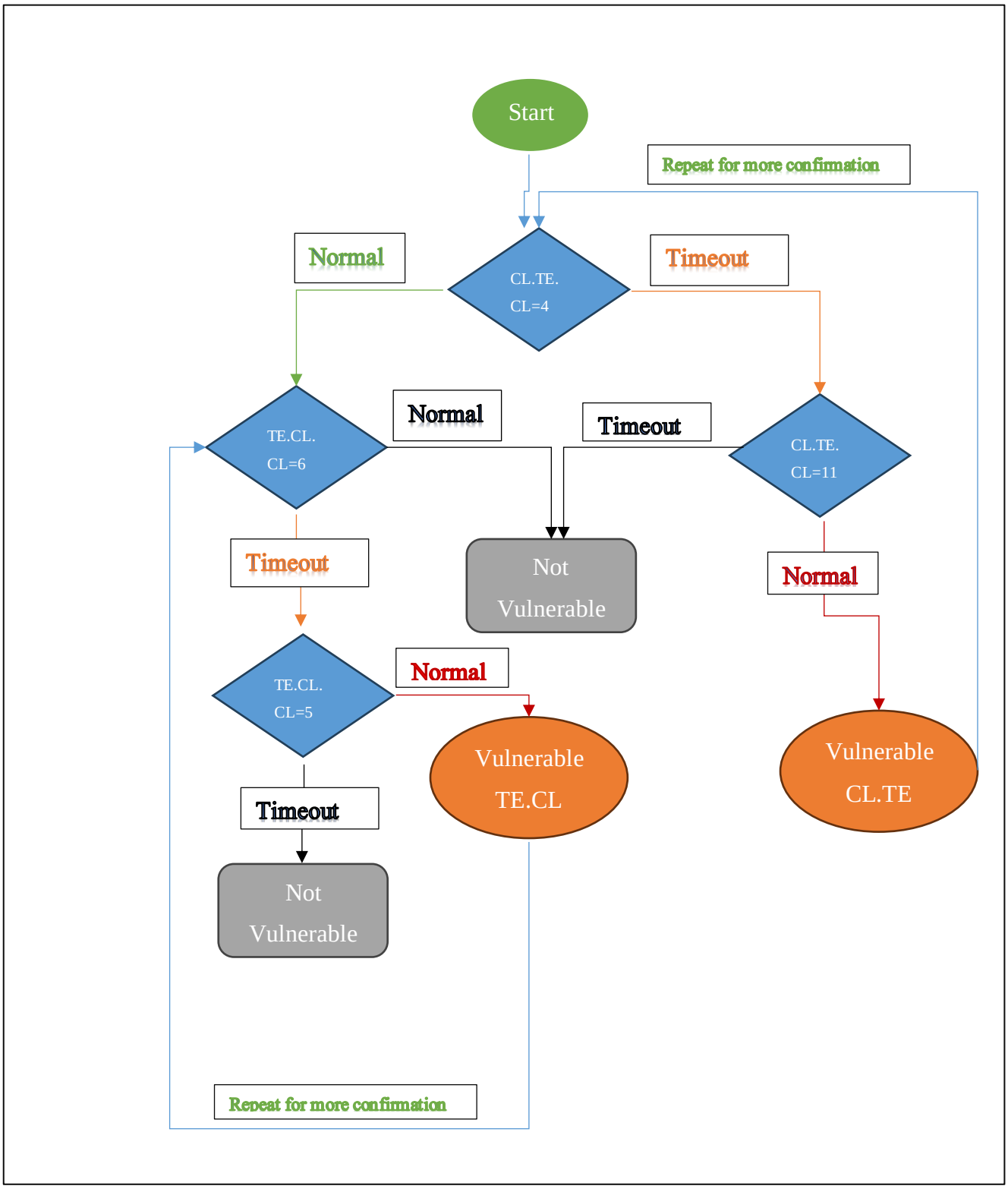


Figure 14 HRS Tool Flowchart

Below is a detailed, step-by-step explanation of the flow chart representing the logic within `detector/`—the core engine for detecting HTTP Request Smuggling vulnerabilities. The flow chart's primary objective is to determine if a given server configuration is susceptible to either of the two main variants of request smuggling vulnerabilities: **CL.TE** (Content-Length then Transfer-Encoding) or **TE.CL** (Transfer-Encoding then Content-Length).

Key Concepts in the Flow Chart

1. **CL= n:**
   This notation indicates that the tool is crafting a HTTP request with a specific `Content-Length` header set to `n`. The chosen `n` is a deliberate mismatch or an unusual value used to test how the server handles discrepancies i.e. would the server timeout or it will send response in normal time.

2. **TE (Transfer-Encoding: chunked):**
   Setting `Transfer-Encoding: chunked` or its other mutations such as `Transfer-Encoding: xchunked` , `Transfer-Encoding: identity,chunked` , `Transfer-Encoding : chunked` and many more of similar type. The detection engine will take one of the TE header first and then it will start its process as shown in the flowchart.

3. **Normal vs. Timeout:**

   – **Normal:** Indicates that the request received a response within the expected timeframe without timing out. This usually suggests that the server accepted and parsed the request in a standard manner. It can also include server rejecting the malformed request and returning a 400 Bad Request error to the detection engine but still server will give the response back within the timeout range.

   – **Timeout:** Indicates that the server failed to respond with in the timeout value set by the detection engine (default is 5 seconds) i.e. it got "stuck" waiting for the request to arrive completely. This usually suggest that there is a chance of finding HRS vulnerability.

Step-by-Step Flow Explanation

1. **Start:**
   The process begins (green start node). The tool is ready to perform its tests against the target server (http or https). Here the tool starts with a list of mutations of TE header as a template and iterate through all of them.

2. **First Decision Point (CL.TE):**
   The tool sends a carefully crafted request with a specific `Content-Length: 4` along with "`Transfer-Encoding: chunked`" or other mutation of TE header but for each start iteration it is fixed.

The detection engine will first try to detect for CL.TE type of HRS i.e. the diamond box shown at the centre will execute first. There is a one reason to start for CL.TE first i.e. If the server is vulnerable to TE.CL and we try to send a payload for CL.TE then it will poison the socket and some random user in production server will get the poisoned response. Hence to avoid that risk, the tool will first perform CL.TE and If it is not found then it will start detection for TE.CL.

– If the response is **Normal**, it means the server responded without abnormal delays, indicating the sever is not vulnerable to CL.TE type of vulnerability and the process moves to the left branch of the flow chart and starts its detection process for TE.CL type of HRS.

– If the response leads to a **Timeout**, it suggests the server waited for the request to complete but since the detection tool did not send number of bytes in body that it mentioned in the CL header of the request, raising suspicion of a CL.TE smuggling scenario.

– If above request timeout, then a next request is made with correct CL header value and if the response is normal for it means that CL.TE vulnerability exist in the server. If this next request also timeout, then this suggest the CL.TE is not present in the system and the detection engine move to search for TE.CL type of HRS i.e. the execution flow moves to the left of the chart.

Raw HTTP request for CL.TE detection:

Timeout Request:

```
POST / HTTP/1.1\r\n
Host: vulnerable.com\r\n
User-Agent: HRS Client\r\n
Content-type: application/x-www-form-urlencoded; charset=UTF-8\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 4\r\n
\r\n
1\r\n
Z\r\n
0\r\n
\r\n
```

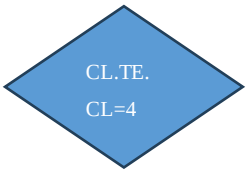


Figure 15 CL.TE Detection Raw Req 1

As we can see in the above raw request, the actual size of the body of the request is 11 bytes but the CL header is set to 4 bytes. If the front-end sever will consider CL header and neglect the TE header then only four bytes of the body will be transferred to the back-end sever i.e. "1\r\n\Z" and if the back-end not considers the TE header then the server will wait for the "0\r\n\r\n" terminating chunk. Since the front-end did not forward the rest of the request body and hence the back-end will timeout. This timeout will suggest a potential for CL.TE type of HRS.

Now to affirm this vulnerability, the tool will make one more request with valid CL value.

Normal Request:

```
POST / HTTP/1.1\r\n
Host: vulnerable.com\r\n
User-Agent: HRS Client\r\n
Content-type: application/x-www-form-urlencoded; charset=UTF-8\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 11\r\n
\r\n
1\r\n
Z\r\n
0\r\n
\r\n
```

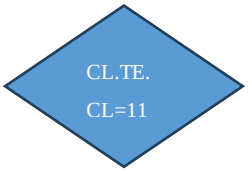


Figure 16 CL.TE Detection Raw Req 2

If the above request is Normal then CL.TE is tentatively confirmed, now the tool will perform same process in again to strongly confirm it.

And if this request is also timeout, then CL.TE vulnerability does not exist and we halt the current iteration and starts again with next TE header mutation.

This CL.TE detection is exclusively handled by **cl_te_detector.py**

3. **Second Decision Point (TE.CL):**

Raw HTTP request for TE.CL detection:

If the initial test with `CL=4` returns Normal response, the tool moves into testing scenarios for TE.CL vulnerability by changing the request body and CL header value (as shown in left of the flowchart).

Timeout Request:

```
POST / HTTP/1.1\r\n
Host: vulnerable.com\r\n
User-Agent: HRS Client\r\n
Content-type: application/x-www-form-urlencoded; charset=UTF-8\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 6\r\n
\r\n
0\r\n
\r\n
```

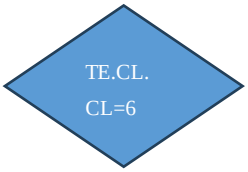


Figure 17 TE.CL Detection raw Req. 1

- The tool tries a scenario represented by `TE.CL`. This means the tool sets "`Transfer-Encoding: chunked`" and `Content-Length: 6` to see if the server misinterprets the request boundaries.
- If this test leads to a **Timeout**, it indicates that the server is vulnerable to TE.CL type of HRS.
- Now to affirm this vulnerability, the tool will make one more request with valid CL value.

Normal Request:

```
POST / HTTP/1.1\r\n
Host: vulnerable.com\r\n
User-Agent: HRS Client\r\n
Content-type: application/x-www-form-urlencoded; charset=UTF-8\r\n
Transfer-Encoding: chunked\r\n
Content-Length: 5\r\n
\r\n
0\r\n
\r\n
```

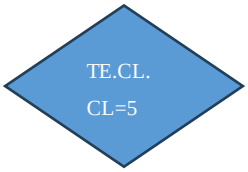


Figure 18 TE.CL Detection Raw Req. 2

- If the `CL=5` test returns **Normal**, the tool concludes a **Vulnerable TE.CL** scenario. This means the server's handling of `Transfer-Encoding` and `Content-Length` together in this particular order (TE before CL) is flawed.
- If the `CL=5` test leads to a **Timeout**, then the server is deemed **Not Vulnerable** under TE.CL conditions.

4. **Repeat for More Confirmation:**

   The tool may re-run tests and apply additional heuristics to ensure reliability. Multiple iterations help confirm whether detected conditions are consistent or just one-off anomalies due to network flakiness or server latency.

- **Left branch (starting from a Normal result with CL=4):** Focuses on detecting **CL.TE** vulnerabilities by incrementally tweaking `Content-Length` and `Transfer-Encoding`.
- This CL.TE detection is exclusively handled by **cl_te_detector.py**
- **Right branch (starting from a Timeout result with CL=6):** Focuses on detecting **TE.CL** vulnerabilities by applying different `Content-Length` values and watching how the server
- This CL.TE detection is exclusively handled by **te_cl_detector.py**

This flow chart outlines a methodical, step-by-step detection process. By tuning the `Content-Length` and `Transfer-Encoding` headers and observing responses (Normal vs. Timeout), the above two scrips logic deduces whether the server suffers from request smuggling vulnerabilities of the CL.TE or TE.CL variety. In doing so, it provides a structured way to identify subtle parsing discrepancies that can lead to dangerous attacks.

3.1.3 HRS Detector workflow for H2.CL and H2.TE

The logic used by H2.CL is implemented in **h2_cl_detector.py** and the logic used to detect H2.TE is implemented in **h2_te_detector.py**

The Flow chart for this detection method would be almost similar to CL.TE and TE.CL with only the difference how the Content-Length and Transfer-Encoding headers are provided and how the body for the request is generated.

For these two techniques to work there is no need to follow the order as in the case of CL.TE and TE.CL.
Each of the above script can be run independently to scan for those categories of bugs.

H2.CL Detection

The **h2_cl_detector.py** script uses a concept of header placement along with header mutations. These header placements are provided from a JSON data structure.

```
        # Define Content-Length header placements to check
        mutations = [
            # 1. Normal header
            {
                "description": "Standard Content-Length header",
                "header_name": "content-length",
                "header_value": "4",
                "type": "normal_header"
            },
            # 2. In custom header value
            {
                "description": "H2.CL via Request Header Injection in Custom
Header Value",
                "header_name": "x-custom",
                "header_value": "foo\r\ncontent-length: 4",
                "type": "custom_header_value"
            },
            # 3. In custom header name
            {
                "description": "H2.CL via Request Header Injection in Custom
Header Name",
                "header_name": "x-custom:foo\r\ncontent-length",
                "header_value": "4",
                "type": "custom_header_name"
            },
            # 4. In Request line
            {
                "description": "H2.CL via Request Line Injection",
                "header_name": ":method",
                "header_value": "POST / HTTP/1.1\r\nContent-Length: 4\r\nx: x",
                "type": "request_line"
            }
        ]
```

The script tries to inject Content-Length: 4 Header in different location of raw http2 binary request format. It tries first in “Standard Content-Length header” then is tries in “H2.CL via Request Header Injection in Custom Header Value” and then “H2.CL via Request Header Injection in Custom Header Name” and finally in “H2.CL via Request Line Injection”

The above script uses our custom http2 client which is designed to send non-RFC compliant request.

As shown in header placement json data CL is set to 4 and the body for the detection request is set to "ABC". The body is one byte short and if the server is vulnerable to H2.CL the server will timeout waiting for the final byte to arrive.
The tool will again send a confirmation request with the same header placement and CL length to 4 but this time the body is set "ABCD", if this response arrives within the timeout seconds, then it is confirmed the server is vulnerable to H2.CL.

H2.TE Detection

The **h2_te_detector.py** script uses a concept of header placement along with header mutations. These header placements are provided from a JSON data structure.

```
 # Define Transfer-Encoding header mutations to test
        mutations = [
            # 1. Normal header
            {
                "description": "Standard Transfer-Encoding header",
                "header_name": "transfer-encoding",
                "header_value": "chunked",
                "type": "normal_header"
            },
            # 2. In custom header value
            {
                "description": "H2.TE via Request Header Injection in Custom
Header Value",
                "header_name": "x-custom",
                "header_value": "foo\r\ntransfer-encoding: chunked",
                "type": "custom_header_value"
            },
            # 3. In custom header name
            {
                "description": "H2.TE via Request Header Injection in Custom
Header Name",
                "header_name": "x-custom:foo\r\ntransfer-encoding",
                "header_value": "chunked",
                "type": "custom_header_name"
            },
            # 4. In Request line
            {
                "description": "H2.TE via Request Line Injection",
                "header_name": ":method",
                "header_value": "POST / HTTP/1.1\r\nTransfer-encoding:
chunked\r\nx: x",
                "type": "request_line"
            }
        ]
```

This follows the same format only the Content-Length: 4 is replaced by Transfer-Encoding: chunked.

Here the detection step sends the body of request as "0\r\n", Terminating chunk without final CRLF. If the server is vulnerable, it will time out for waiting for final "\r\n". Once we detect this timeout will send a confirmation request with the body as "0\r\n\r\n" and if it responds without delay, H2.TE is confirmed.

4. Theoretical & Empirical Result Analysis

4.1 HRS Detector Overview

HTTP Request Smuggling Detection Tool (hrs_detector)

A Python toolkit for detecting HTTP request smuggling vulnerabilities, including modern HTTP/2-based variants. It also includes a web-based GUI for easier interaction.

Overview

This tool is designed to detect various types of HTTP request smuggling vulnerabilities:

- CL.TE - Content-Length / Transfer-Encoding desync
- TE.CL - Transfer-Encoding / Content-Length desync
- H2.TE - HTTP/2 to HTTP/1 Transfer-Encoding desync
- H2.CL - HTTP/2 to HTTP/1 Content-Length desync

The tool features custom HTTP/1.1 and HTTP/2 clients that allow sending non-RFC-compliant requests required for detecting these vulnerabilities.

Installation

You can install the HTTP Request Smuggling Detection Tool in several ways:

1. Install from Source

```
# Clone the repository
git clone https://github.com/yourusername/hrs_finder.git
cd hrs_finder

# Install as a package
pip install .
```

With an editable installation (-e flag), any changes you make to the source code will be immediately available without reinstalling.

Usage

This tool utilizes predefined header variations stored in JSON format within the payloads/ directory (te_headers.json, cl_headers.json) for constructing test requests.

Scan Command

The unified scan command allows testing for all supported vulnerability types in a single run:

```
# Scan for all supported vulnerability types
hrs_detector scan https://example.com

# Scan for specific types (comma-separated, no spaces)
hrs_detector scan https://example.com --type cl.te,te.cl

# Scan for specific types (comma-separated with spaces, requires quotes)
hrs_detector scan https://example.com --type "cl.te, te.cl"

# Scan with custom headers
hrs_detector scan https://example.com -H "Cookie: session=1234" -H "X-Forwarded-
For: 127.0.0.1"

# Scan with verbose output
hrs_detector scan https://example.com -v

# Scan with output to a JSON file
hrs_detector scan https://example.com --output results.json

# Stop scanning after the first vulnerability is found
hrs_detector scan https://example.com -e
# or
hrs_detector scan https://example.com --exit-first

# Specify payload placement for H2.CL/H2.TE (e.g., smuggle via header name)
hrs_detector scan https://example.com --type h2.cl,h2.te --h2-payload-placement
custom_header_name
```

Web GUI

A web-based graphical user interface is available in the frontend directory. It allows you to run scans and view results through your browser.

```
# Navigate to the frontend directory
cd frontend

# Install GUI dependencies
Pip3 install fastapi uvicorn websockets pydantic

# Start the server from the frontend directory
Python3 server.py
```

4.2 Tools and Technologies Used

hrs_detector /

- ├── frontend/ # Web GUI Application
- │ ├── static/ # CSS, JavaScript, Images
- │ ├── templates/ # HTML Templates
- │ ├── server.py # FastAPI Backend for GUI
- │ ├── README.md # Frontend specific documentation
- │ └── project_content.md # Frontend specific documentation
- ├── src/ # Main source code
- │ ├── __init__.py
- │ ├── cli/ # Command-line interface
- │ │ ├── __init__.py
- │ │ └── main.py
- │ ├── clients/ # HTTP clients
- │ │ ├── __init__.py
- │ │ ├── base.py # Base client interface
- │ │ ├── http1.py # HTTP/1.1 client using asyncio
- │ │ └── http2.py # HTTP/2 client using h2
- │ ├── detectors/ # Vulnerability detector modules
- │ │ ├── __init__.py
- │ │ ├── cl_te_detector.py
- │ │ ├── te_cl_detector.py
- │ │ ├── h2_cl_detector.py
- │ │ └── h2_te_detector.py
- │ └── utils/ # Utility functions
- │ ├── __init__.py
- │ ├── tls.py
- │ └── logging.py
- ├── examples/ # Example usage scripts (if any)
- ├── payloads/ # Test payloads (header variations)
- │ ├── te_headers.json # Transfer-Encoding variations
- │ └── cl_headers.json # Content-Length variations
- ├── tests/ # Unit and integration tests
- ├── main.py # Direct execution wrapper
- ├── hrs_finder.sh # Shell script wrapper for main CLI
- ├── setup.py # Package installation script
- ├── requirements.txt # Package dependencies
- ├── test_http2_client.py # Standalone HTTP/2 client test script
- ├── README.md
- ├── project_content.md

Implemented Components

src/clients/base.py

- BaseClient abstract base class defining the interface for HTTP clients
- Provides common methods and properties that both HTTP/1.1 and HTTP/2 clients must implement
- Methods:
  - connect() - Establish a connection to the target server
  - close() - Close the connection to the target server
  - send_request() - Send an HTTP request and return the response
  - send_raw() - Send raw bytes over the connection
  - receive_raw() - Receive raw bytes from the connection

src/clients/http1.py

- HTTP1Client implementation of the BaseClient interface for HTTP/1.1
- Uses asyncio for asynchronous I/O
- Features:
  - Full control over request construction
  - Support for non-RFC-compliant requests
  - Detailed logging of requests and responses
  - TLS support with configurable options
  - Connection reuse (keep-alive)
  - Raw bytes mode for complete control

src/clients/http2.py

- HTTP2Client implementation of the BaseClient interface for HTTP/2
- Uses the h2 library for HTTP/2 framing
- Features:
  - Support for sending malformed headers and data frames
  - Ability to send duplicate pseudo-headers
  - Support for conflicting content-length values
  - Control over DATA frames (partial bodies, withheld termination)
  - TLS with ALPN negotiation
  - Enhanced verbose logging with complete raw HTTP/2 request details
  - Detailed frame-level logging when verbose mode is enabled
  - Binary data logging with hex dumps when necessary
  - Timestamps, file names, and line numbers in log messages

src/cli/main.py

- Main command-line interface for the tool
- Commands:
  - scan - Run multiple detector types at once
  - request - Send a custom HTTP/1.1 request
- Uses the click library for the CLI
- Features:
  - Unified scanning interface for all detector types
  - Custom headers support
  - Output to JSON for further analysis
  - Verbose logging option for detailed output
  - Support for comma-separated vulnerability types

src/detectors/cl_te_detector.py

- Detector for CL.TE HTTP request smuggling vulnerabilities
- Uses time-delay technique to detect vulnerabilities
- Features:
  - Support for multiple Content-Length and Transfer-Encoding header variations
  - Detailed output with raw request and response

src/detectors/te_cl_detector.py

- Detector for TE.CL HTTP request smuggling vulnerabilities
- Uses time-delay technique to detect vulnerabilities
- Features:
  - Support for multiple Transfer-Encoding header variations
  - Detailed output with raw request and response

src/detectors/h2_cl_detector.py

- Detector for H2.CL HTTP request smuggling vulnerabilities
- Tests HTTP/2 to HTTP/1.1 Content-Length desync
- Sends HTTP/2 requests with a Content-Length header value larger than the actual body
- Features:
  - Advanced logging with detailed frame information (when verbose mode is enabled)
  - Multiple header placement options:
    - normal_header: Standard Content-Length header
    - custom_header_value: Content-Length smuggled in a custom header value
    - custom_header_name: Content-Length smuggled in a custom header name
  - Configurable timeout for detection
  - Detailed vulnerability reporting with timing ratios and detection explanation

src/detectors/h2_te_detector.py

- Detector for H2.TE HTTP request smuggling vulnerabilities
- Tests HTTP/2 to HTTP/1.1 Transfer-Encoding desync
- Sends HTTP/2 requests with a Transfer-Encoding: chunked header and an incomplete chunk
- Features:
    - Advanced logging with detailed frame information (when verbose mode is enabled)
    - Multiple header placement options:
        - normal_header: Standard Transfer-Encoding header
        - custom_header_value: Transfer-Encoding smuggled in a custom header value
        - custom_header_name: Transfer-Encoding smuggled in a custom header name
    - Configurable timeout for detection
    - Detailed vulnerability reporting with timing ratios and detection explanation
    - Uses "0\r\n" as the body instead of "abc" (like in H2.CL)
    - Default end_stream=False to keep the connection open for timing-based detection

src/utils/tls.py

- TLS utilities for HTTP clients
- Functions:
    - create_ssl_context() - Create an SSL context with appropriate settings for HTTP/1.1 and HTTP/2
    - Support for disabling SSL verification
    - ALPN protocol negotiation for HTTP/2

src/utils/logging.py

- Logging utilities for the tool
- Functions:
    - setup_logging() - Configure logging for the tool
    - get_logger() - Get a logger instance
- Features:
    - Configurable log level (debug, info, warning, error)
    - Console and file logging
    - Rich formatting for console logs
    - Timestamps in log messages
    - File names and line numbers in log messages
    - Clickable file paths in supported terminals

4.3 Scanning Sample Applications

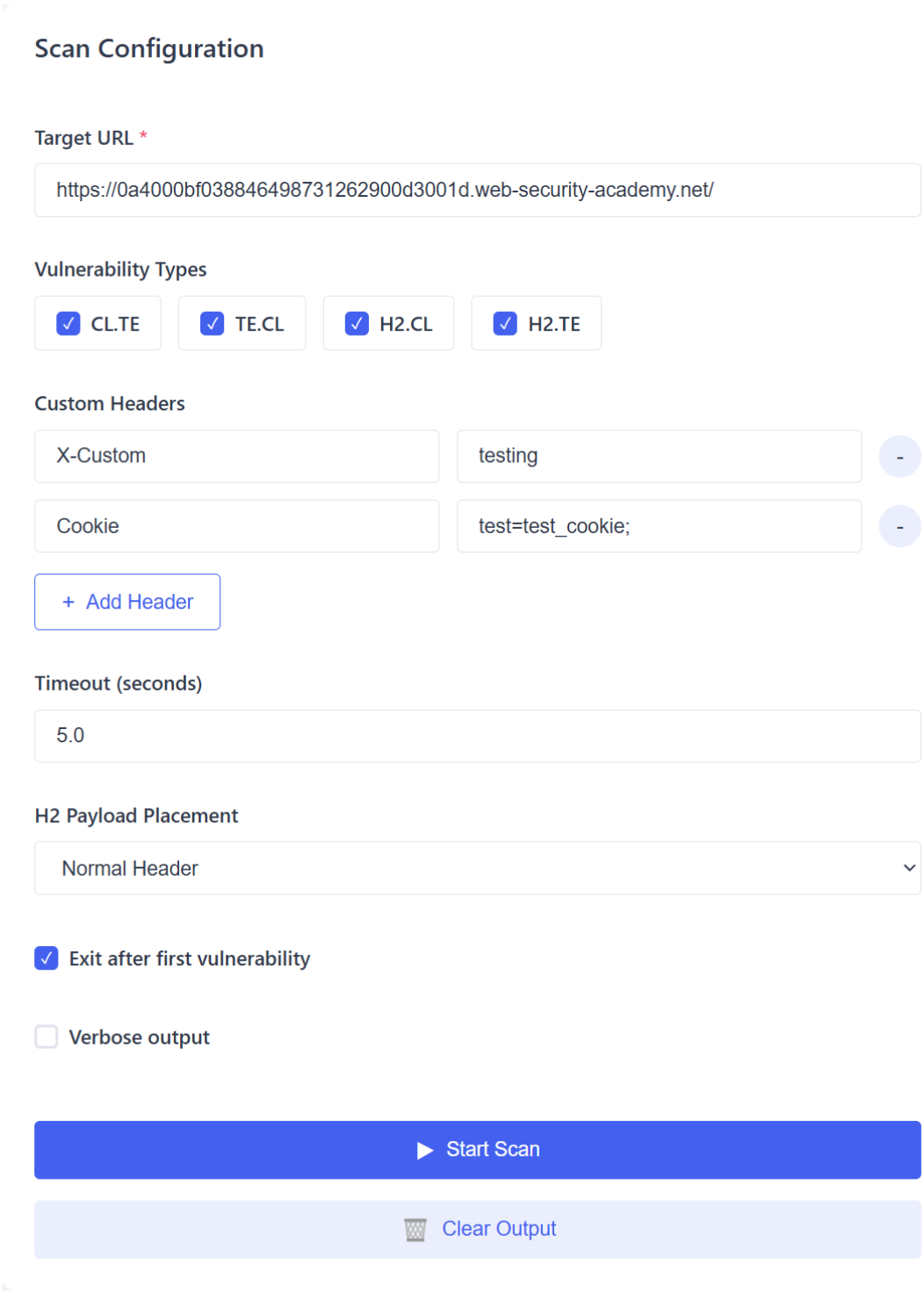


Figure 19 HRS Detector Scan Configuration UI

1. Start the server:

```
Python3 server.py
```

2. Open your browser and navigate to http://localhost:8000
3. Configure your scan:
    - Enter the target URL
    - Select vulnerability types to scan for
    - Add any custom headers
    - Configure additional options
4. Click "Start Scan" to begin the scan
5. View real-time results in the output area
6. Review automatically extracted vulnerability findings in the dedicated table below

Configuration Options

- **Target URL**: The URL to scan
- **Vulnerability Types**: CL.TE, TE.CL, H2.CL, H2.TE
- **Custom Headers**: Add any number of custom headers
- **Timeout**: Specify request timeout in seconds
- **H2 Payload Placement**: (Only when H2 types selected) Configure where payload is placed
- **Exit After First Vulnerability**: Stop scan after finding first vulnerability
- **Verbose Output**: Show detailed output from the scanner

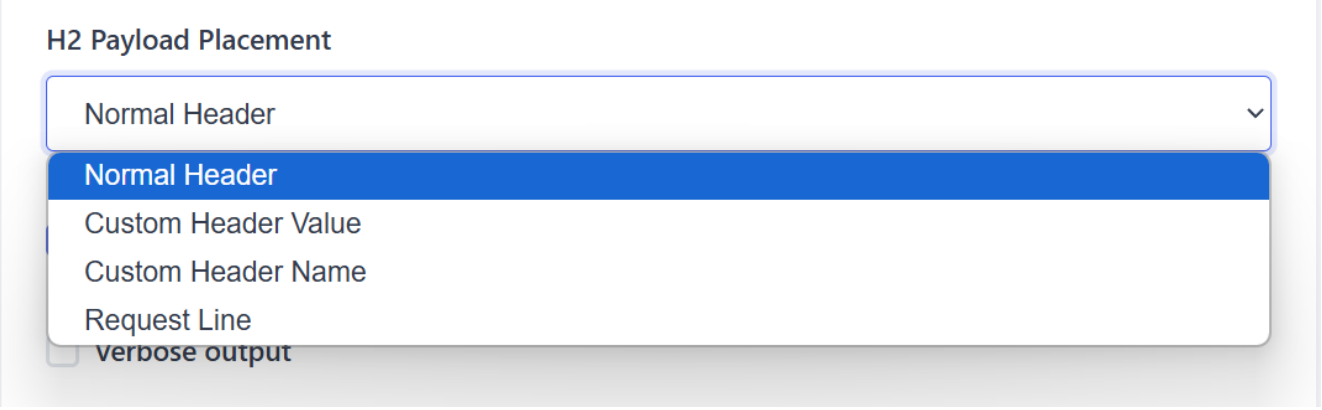


Figure 20 HRS Detector Header Placements

Scan Results Area

The output area displays real-time scan results with proper formatting for different message types (standard logs, errors, debug messages). A scrollbar appears when content exceeds the container size.

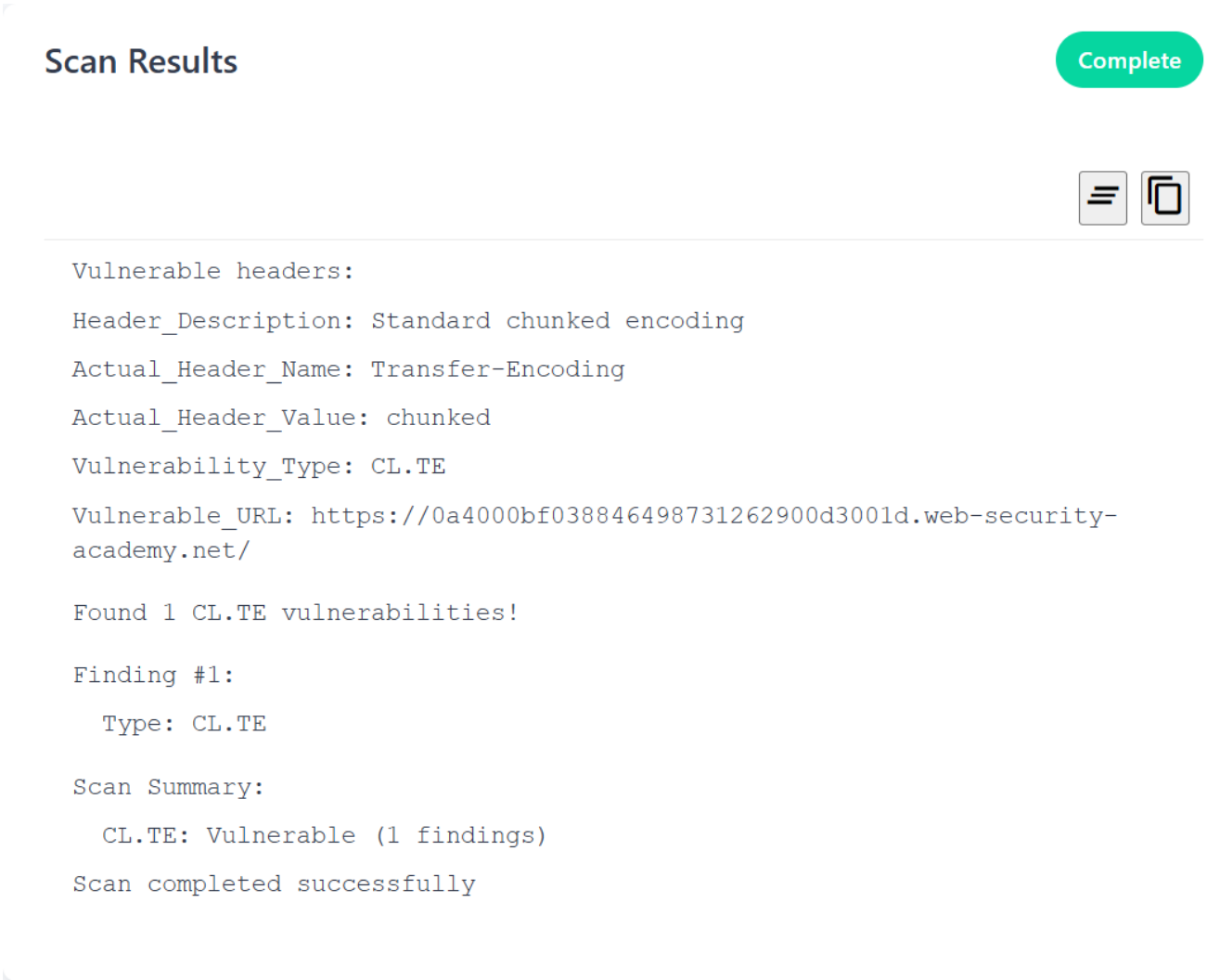


Figure 21 HRS Detector Scan Results Console Area

Vulnerability Findings Table

The findings table automatically extracts and presents vulnerability information in a structured format with columns for:

- URL
- Vulnerability Type
- Description
- Header Name
- Header Value

Each table has separate controls:

- **Reset Findings**: Clear just the vulnerability findings table
- **Clear Output**: Clear just the scan output area

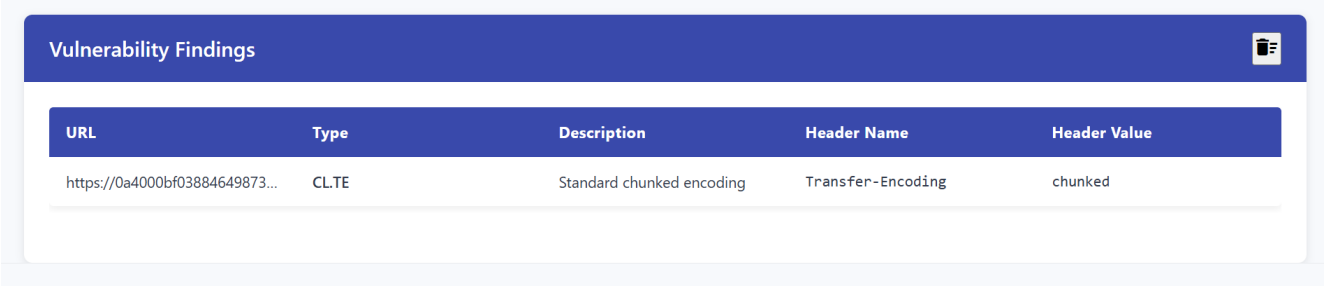


Figure 22 HRS Detector Vulnerability Findings Table

Architecture:

- **Frontend**: HTML5, CSS3, and vanilla JavaScript
- **Backend**: FastAPI Python server
- **Communication**: WebSockets for real-time updates

5. Conclusion

This research project explored the complex domain of **HTTP Request Smuggling (HRS)** vulnerabilities, a critical and increasingly relevant concern in modern web application security. Through comprehensive analysis, identification of diverse attack vectors, and the development of an automated detection tool, this study has enhanced the understanding of HRS vulnerabilities and provided a practical approach for mitigating associated risks.

The investigation began with a detailed exploration of the HTTP protocol's evolution, from the foundational HTTP/1.0 to the advanced HTTP/2 standards. This historical context highlighted how architectural improvements and performance optimizations, though beneficial, inadvertently introduced new avenues for exploitation. An in-depth review of modern web architectures and the subtle intricacies involved in parsing HTTP requests was essential in identifying how discrepancies arise, allowing attackers to exploit these gaps by injecting malicious requests.

Central to this research was the systematic analysis of various HRS vulnerability types, including **CL.TE, TE.CL, CL.CL, TE.TE, CL.0, H2.CL, and H2.TE**. By carefully examining each variant, the study clarified how mismatches in header interpretations—particularly involving the **Content-Length** and **Transfer-Encoding** headers and **HTTP/2 to HTTP/1.1. downgrading** —can cause request desynchronization and enable attackers to smuggle harmful payloads through intermediary systems.

Addressing the complexity of detecting such vulnerabilities, this project introduced the **HRS Detector tool**, a core contribution aimed at simplifying and improving detection methodologies. Unlike traditional manual approaches, the tool offers automation paired with flexibility. Its user-friendly graphical interface, configurable scanning modes, and integrated request repeater significantly streamline vulnerability identification and validation. The structured detection logic, clearly represented through intuitive flowcharts, ensures methodical and thorough analysis, enhancing the reliability of results. Furthermore, the integrated repeater capability allows users to manually inspect and re-analyze detected requests, adding versatility to the tool.

In conclusion, this project contributes substantially to the understanding and mitigation of HTTP Request Smuggling vulnerabilities. By combining rigorous theoretical analysis with the development of a practical detection solution, it underscores both the subtle dangers posed by HRS and the effectiveness of targeted, automated detection methods. The **HRS Detector** not only addresses existing challenges in web security testing but also lays a strong foundation for future research and tool enhancement. Ultimately, this work significantly advances the effort to safeguard web applications against a critical, evolving threat, benefiting organizations, security professionals, and end-users alike.

6. Future Scope

This project has established a robust foundation for addressing **HTTP Request Smuggling (HRS)** vulnerabilities through the development of the **HRS Detector tool**. However, given the dynamic and constantly evolving nature of web technologies and associated security threats, ongoing improvements and enhancements are necessary. To ensure the continued effectiveness and relevance of the tool, several key areas have been identified for future development:

1. Enhanced Detection Logic:

While the current detection logic is effective, there is scope to further refine and enhance its accuracy and efficiency. Integrating additional heuristics or adopting machine learning techniques could significantly reduce false positives and negatives, enhancing the reliability and precision of vulnerability detection. Expanding detection coverage to include more variations of known attack vectors, as well as developing mechanisms for emerging HRS attack techniques, will further increase the tool's comprehensiveness. These improvements will ensure broader detection capabilities without negatively impacting performance.

2. Expanded Integration and Automation:

Integrating the **HRS Detector** with existing security testing frameworks and Continuous Integration/Continuous Deployment (CI/CD) pipelines would greatly improve its utility and ease of adoption. Such integration would enable security teams to incorporate HRS vulnerability scanning seamlessly into routine security assessments and development workflows. Additionally, providing robust API support will facilitate easier interoperability with third-party systems, enhancing the tool’s accessibility and automation potential.

7. Bibliography

1. **Chaim Linhart, Amit Klein, Ronen Heled, Steve Orrin. "HTTP Request Smuggling."** https://cgisecurity.com/lib/http-request-smuggling.pdf

2. **Nicolas Grégoire. "Hiding Wookiees in HTTP."** https://media.defcon.org/DEF%20CON%2024/DEF%20CON%2024%20presentations/DEF%20CON%2024%20-%20Regilero-Hiding-Wookiees-In-Http.pdf

3. **James Kettle. "HTTP Desync Attacks: Request Smuggling Reborn."** https://portswigger.net/research/http-desync-attacks-request-smuggling-reborn

4. **Amit Klein. "HTTP Request Smuggling in 2020: New Variants, New Defenses and New Challenges."** https://i.blackhat.com/USA-20/Wednesday/us-20-Klein-HTTP-Request-Smuggling-In-2020-New-Variants-New-Defenses-And-New-Challenges.pdf

5. **Jake Miller. "Request Smuggling Via HTTP/2 Cleartext [h2c]."** https://i.blackhat.com/USA-20/Wednesday/us-20-Klein-HTTP-Request-Smuggling-In-2020-New-Variants-New-Defenses-And-New-Challenges.pdf

6. **James Kettle. "Browser-Powered Desync Attacks: A New Frontier in HTTP Request Smuggling."** https://portswigger.net/research/browser-powered-desync-attacks

8. Plagiarism Report

VIPIN-MTCS58-MAJOR-PROJECT-Continued

National Forensic Sciences University

Document Details

Submission ID
trn:oid:::28903:94511044

Submission Date
May 6, 2025, 10:09 AM GMT+5:30

Download Date
May 6, 2025, 10:15 AM GMT+5:30

File Name
VIPIN-MTCS58-MAJOR-PROJECT-Continued

File Size
80.9 KB

71 Pages

11,408 Words

69,253 Characters

NFSU

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **16** Not Cited or Quoted 7%
  Matches with neither in-text citation nor quotation marks
- **5** Missing Quotations 0%
  Matches that are still very similar to source material
- **6** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
4% Publications
12% Submitted works (Student Papers)